Graphical Abstract

Linearizability can be Asynchronous Distributed Runtime Verified with non-linearizable objects

Gilde Valeria Rodríguez, Armando Castañeda

Highlights

Linearizability can be Asynchronous Distributed Runtime Verified with non-linearizable objects

Gilde Valeria Rodríguez, Armando Castañeda

- We present an asynchronous distributed communication interface for runtime verification of linearizability, replacing atomic snapshots with non-linearizable collects and enabling implementations based only on read/write operations or/and $Fetch\&Add$.

- Our model detects execution intervals and enables reasoning about discrepancies between the actual execution and the detected one, bridging the gap between theoretical guarantees and practical verification.

Linearizability can be Asynchronous Distributed Runtime Verified with non-linearizable objects

Gilde Valeria Rodríguez[a,*], Armando Castañeda[b]

[a]Posgrado en Ciencia e Ingeniería de la Computación, UNAM, Universidad Nacional Autónoma de México, Ciudad Universitaria, Mexico City, 04510, Mexico
[b]Instituto de Matemáticas, UNAM, Universidad Nacional Autónoma de México, Ciudad Universitaria, Mexico City, 04510, Mexico

Abstract

In distributed runtime verification, processes communicate to capture the real-time execution of an algorithm $\mathcal{A}$ (communication interface) and produce a correctness verdict (verification interface). In this work, we focus on design a distributed communication interface to runtime verify linearizability. Since linearizability depends on the real-time ordering of operations, recently, was introduced the first model capable of detecting the execution intervals of operations in a current execution of any algorithm under inspection in a distributed and asynchronous setting. This resulted in a complete and sound verification approach. However, the model remained highly theoretical, as it relied on a snapshot object abstraction, limiting its practicality. We prove that this model can lead to efficient and simple implementations, effectively bridging the gap between the theoretical model and a practical approach. This is achieved by replacing snapshots with collect objects, which read all array entries in a non-atomic manner, even though linearizability is typically regarded as an atomic property. We present a Verifier $\mathcal{V}^+$ that performs runtime verification of any algorithm $\mathcal{A}$ by transforming it into a Wrapper $\mathcal{A}^+$, which produce responses to invocations of $\mathcal{A}$ and return information about its current execution. Both $\mathcal{V}^+$ and $\mathcal{A}^+$ operate exclusively using collect and read/write operations. The verifier indirectly runtime verifies $\mathcal{A}$ through $\mathcal{A}^+$, this form of indirect verification arises from the impossibility of directly verifying any implementation $\mathcal{A}$ in a fully distributed and asynchronous setting. To demonstrate the feasibility of a practical and efficient implementation, we introduce $\mathcal{B}^+$, a runtime-verifiable algorithm that, by allowing the collect abstraction, enables the use of a simple atomic $Fetch\&Add$ operation, which can also be a relaxed $Fetch\&Add$ using only read/write operations. Despite its simplicity, $\mathcal{B}^+$ effectively retains the verification guarantees of the model. Finally, we argue that this indirect runtime verification problem inherently requires atomicity and cannot be addressed using a weaker memory model, such as sequential consistency. While we successfully replace snapshots with collects, the detection of the real-time order of operations still demands atomicity.

Keywords: Runtime Verification, Distributed, Asynchronous, Linearizability, Atomicity, communication interface, concurrent shared memory algorithms

1. Introduction

1.1. Context

The runtime verification is a formal verification technique with very practical goals: it aims to ensure the correctness of a system during its execution by observing its runtime behavior [1]. Runtime verification frameworks are usually divided in two layered systems: the *communication interface*, responsible for detecting and describing the current execution and enabling information flow from the underlying system to the *verification interface* which determines if the current execution its correct with respect to a correctness condition [2].

We focus on the runtime verification of *concurrent shared-memory algorithms*, where multiple processes interact through a shared memory. Moreover, we aim to runtime verify in a distributed and asynchronous manner, meaning that the processes share information about the underlying concurrent shared-memory algorithm being verified and this processes run at different speeds; which add a step of difficult due to nondeterminism, and possible execution interleavings.

The runtime verification of linearizability [3], a fundamental correctness condition for concurrent algorithms, has already been studied from different perspectives. One approach focuses on developing analyses and methods to determine whether an execution is linearizable [4, 5, 6, 7, 8]. This approach can be regarded as the verification interface, often leaving the detection/communication interface implicit or, when taking a practical approach, generating all possible or a significant number of interleavings [5, 9, 10]. This approach is challenging, as checking whether a single execution is linearizable has been proven to be **NP**-*complete* [4], though polynomial solutions exist for specific implementations of certain objects [6, 8].

Another approach focuses on detecting the current execution of the algorithm under inspection for runtime verification, can be considered as the communication interface, in this work we focus on designing a distributed communication interface, leaving implicit the verification interface. In [11, 12], a snapshot

*Corresponding author
Email addresses: gilde@ciencias.unam.mx (Gilde Valeria Rodríguez), armando.castaneda@im.unam.mx (Armando Castañeda)

abstraction approach is used to detect the current execution in a distributed setting. However, both works remain highly theoretical due to the reliance on snapshots. The setting in [12] is fully asynchronous and fault tolerant, in contrast to [11]. More details on [12, 11] are provided in the related work section.

Nevertheless, this approach also faces fundamental challenges, as it is impossible to detect the current execution of any algorithm $\mathcal{A}$ in an asynchronous and distributed setting for runtime verify the linearizability of $\mathcal{A}$ [12]. Even if the model is designed to capture the execution interval of each operation, i.e., the real-time order of invocations and responses in a typical concurent execution [3]. We refer to this property as *interval detection*. Namely, for common objects like pools, queues, stacks, and even consensus, no verifier can simultaneously achieve *soundness* and *completeness*. This impossibility stems from the inherent limitation of determining the real-time order of causally related operations [13].

An alternative to overcoming this impossibility was presented in [12], which introduced the concept of a Wrapper $\mathcal{A}^*$ (classified under the *DRV* class in Section 7 in [12]) to runtime verify linearizability of any algorithm $\mathcal{A}$ *indirectly* and *predictive*. The *Wrapper* $\mathcal{A}^*$ of $\mathcal{A}$ is a concurrent algorithm that "wraps" the execution of $\mathcal{A}$ by augmenting its operations with additional information, enabling the construction of "sketches" that represent the current execution of the $\mathcal{A}^*$, following an *interval detection*.

Based on this concept, a *sound* and *complete* verifier $\mathcal{V}^*$ was presented, to verify indirectly any algorithm $\mathcal{A}$ as a black box, i.e. the specification its unknown. This verifier uses the sketches produced by the Wrapper $\mathcal{A}^*$ to determine whether they are linearizable (fulfilling Definition 2.2). If it is found to be non-linearizable, it implies that the underlying algorithm $\mathcal{A}$ it is also non-linearizable. Thus, the verification of $\mathcal{A}$ is achieved indirectly through $\mathcal{A}^*$. Additionally, if a process detects a violation of linearizability and reports an ERROR, it can provide a *witness*, which is a specific non-linearizable execution of $\mathcal{A}^*$ that substantiates its claim.

1.2. Contribution

In this work, motivated by the goal of leveraging the theoretical model in [12] to design practical implementations, we overcome the limitations of its reliance on snapshot objects by replacing them with collect objects. Unlike snapshot objects, a collect object is a non-linearizable object that enables reading all entries in an array not atomically and there exists simple linear implementations [14].

Snapshots allow all operations in an array to be read atomically, enabling the views obtained by snapshot operations to be ordered by ascending containment [15, 16]. This feature was crucial in [12] for obtaining the wrapper $\mathcal{A}^*$ of any algorithm $\mathcal{A}$ and sharing it with the Verifier. Although snapshot algorithms exhibit good theoretical step complexity bounds, their practical performance remains uncertain. The fastest known algorithms (e.g., [17]) rely on intricate and sophisticated constructions; ; for example, some leverage powerful objects like CAS [18]. In contrast, simpler algorithms often exhibit high step complexity or provide only weak termination guarantees [19].

A key objective in the runtime verification domain is to design communication interfaces that minimally interfere with the system under inspection (e.g.,[20, 21, 22, 23]). Replacing snapshots with collect objects simplifies the communication interface and enhances its potential efficiency.

The key insight enabling the verification of linearizability using non-linearizable objects lies in Lemma 4.1, which establishes that collect objects satisfy the *containment in preceding* property. This property captures the order of preceding operations, aligning with linearizability's requirement to preserve this order. However, while collects do not accurately reflect the order of concurrent operations, atomic snapshot objects, despite their atomic guarantees, are also imprecise capturing concurrent operations in a fully asynchronous setting due to unexpected delays. This limitation stems from the impossibility of determining the real-time order of operations in a fully-asynchronous system [13].

Moreover, by leveraging the non-atomic nature of collect objects to read the global state, we present a runtime-verifiable algorithm that relies on a $Fetch\&Add$ operation, which can be either atomic or relaxed version with multiplicity [24].

Finally, to broaden the discussion, we explore whether atomic operations are inherently necessary for the indirect runtime verification of linearizability or if a weaker memory model could suffice.

1.3. Organization

To this end, we introduce an improved wrapper algorithm, $\mathcal{A}^+$, for verifying any algorithm $\mathcal{A}$, presented in Section 3. In Section 5, we introduce the Verifier $\mathcal{V}^+$, both of which rely solely on collect objects and read/write operations.

In this new setting, the key challenge lies in the Verifier $\mathcal{V}^+$, which must transform the views returned by the collect object into a sketch that describes the *well-formed* current execution of $\mathcal{A}^+$. This transformation, achieved through Algorithm 2 in section 4, constitutes the primary technical contribution of this work.

Moreover, we present a more practical and verifiable implementation: the algorithm $\mathcal{B}^+$ (Section 6), which integrates $\mathcal{A}^+$ and the Verifier $\mathcal{V}^+$ through a *decoupled* collect object. This approach is inspired by *decoupled* snapshot algorithms [25, 18]. In this design, the processes executing the wrapper $\mathcal{A}^+$ (producers) are solely responsible for generating executions, while the processes running the Verifier $\mathcal{V}^+$ (verifiers) focus on obtaining a sketch that represents the current execution of the wrapper and reporting ERROR when the execution is determined to be non-linearizable.

To conclude, in Section 7, we explore whether replacing collects with snapshots allows us to eliminate atomicity entirely, including atomic writes, to potentially improve efficiency. However, we argue that this would compromise correctness, rendering interval detection ineffective. For instance, if the Wrapper $\mathcal{A}^+$ were executed under a sequentially consistent model, the Verifier $\mathcal{V}^+$ would no longer satisfy the required guarantees, underscoring the importance of atomic operations for ensuring correctness in this context.

1.4. Related work

Runtime verification of concurrent shared-memory systems is a challenging task [26, 27, 28, 29] due to the unpredictability of thread execution, which results in a vast number of possible interleavings. This complexity leads to a wide range of errors including *deadlocks*, *data races*, *atomicity violations*, and violations of higher-level correctness properties such as linearizability [3] and sequential consistency [30]. Moreover, there is an inherent tension between correctness and performance in concurrent programming: techniques that enhance performance often make reasoning about correctness significantly harder.

Additionally, performing runtime verification of concurrent shared-memory systems with a distributed communication interface introduces several challenges [2, 31, 11, 32, 12, 33, 34]. One fundamental difficulty is the need to communicate partial views or local verdicts about the system's execution. Some approaches rely on global clocks or locks to coordinate verification in a synchronous manner. However, achieving this in an asynchronous and fault-tolerant setting poses additional challenges, such as discrepancies between the actual execution and the detected behavior, as well as unexpected delays or variations in process speeds [2, 12, 35, 36].

The model considered for this work [12] is, to the best of our knowledge, the only one that proposed a fully asynchonous and fault-tolerant distributed runtime verification framework of a high-level correctness property such as linearizability. Moreover, the definition is based on Pnueli's approach [37], however has the unique feature to provide an analisys of the gap between the detected execution and the actual execution of the algorithm under inspection. This can be achieve by detecting the execution interval of each operation, capturing the input/output pairs with a real-time order relation, *interval detection*. It is a common topic in the runtime verification literature that, for several reasons, the detected execution may be imprecise or even incomplete (see survey [38]). The results in [12] hold for a generalization of linearizability. Here, for clarity, we focus on linearizability, but our results can be extended to that generalization as well.

While the series of works [2, 31, 11] have proposed runtime verification solutions for task solvabity [39], they do not explicitly focus on verifying linearizability. These approaches consider a shared memory system that solves a series of tasks, where the goal is to verify at runtime whether the outputs of each task are correct, i.e. they satisfy the inputs/outputs relation specifying the task. To achieve this, an asynchronous fault-tolerant shared memory algorithm executes whenever a task in the series is solved, with the assumption that the verification process terminates before the processes proceed to the next task. Although these solutions are distributed and fault-tolerant, they are not fully asynchronous. Moreover, while they sample executions of the underlying algorithm under inspection, they do not focus on capturing the real-time order of operations, a crucial aspect in verifying linearizability, as it relies on the real-time ordering of operations.

Prior works have made significant contributions to the runtime verification of linearizability, primarily focusing on developing algorithms to determine whether an execution is linearizable [9, 10, 40, 5, 41, 42], rather than analyzing the distributed aspects of verification. Some of these approaches generate executions in a distributed manner for verification purposes [9, 10, 5, 8, 41]. While these methods consider input/output intervals, they do not analyze discrepancies between the actual execution and the generated execution. Moreover, in most cases, the actual execution is not even generated in an asynchronous setting. Additionally, these methods assume a white-box model, where the implementation code is fully known, whereas our approach treats the algorithm as a black-box, making no assumptions about its internal structure and code.

In [9, 10], additional code is inserted into a *white-box* algorithm to record its execution in a *log* (a sequence of events), which is later verified by a single process. Consequently, this approach is neither fully distributed nor fault-tolerant, as the events recorded in the log require consensus or locking mechanisms [27, 28, 29] to ensure atomicity. Moreover, the relationship between the actual execution of the algorithm under inspection and the execution recorded in the log is not analyzed.

In [5], the authors also study execution intervals, but unlike our approach, they do not use them to capture the real-time order of operations. Instead, they generate executions in a distributed manner without explicitly tracking execution intervals and introduce a parameter k that adjusts the length of operation intervals. By manipulating k, they decrease or increase the complexity of enumerating possible linearizations. While their polynomial-time decision mechanism makes the approach efficient, it is not asynchronous, as the ordering of operations is well-defined, restricting the space of possible interleavings. This may result in missing violations or introducing executions that do not actually occur in the algorithm under verification.

In [41] the authors propose a technique for runtime verification of linearizability by precomputing schemas in a static manner. These schemas represent sets of return-value outcomes that certain implementations admit, referred to as ADT (Abstract Data Type) objects. While this approach is efficient and scalable is not asynchronous or fault-tolerant. The interleavings are precomputed, which limits its applicability to distributed or asynchronous settings where the order of operations cannot be statically determined.

In [40], the authors introduce the *Hindsight Lemma*, a technique for verifying linearizability by infering the existence of a global state where its operation can be linearized, based solely on local atomic observations of the shared state. This approach avoids the need to reason about linearization points in other threads, which is often complex and non-intuitive. While the approach simplifies the verification of *optimistic algorithms*, is not asynchronous nor fault-tolerant It is known that there are linearizable implementations whose linearization points are *not fixed* [43], thus the approach in [9, 10, 40, 42] is not complete.

In [42] the authors propose a systematic approach to deriving concurrency control mechanisms (e.g., locks) from sequential proofs of correctness. This is achieved by identifying potential interference between threads and inserting locks to prevent violations of the assertions. The approach is extended to guarantee linearizability, ensuring that the library behaves like its

sequential specification even in a concurrent setting, however, due to the concurrency control mechanisms the system is not asynchonous, nor fault-tolerant.

2. Distributed Indirect and Predictive Runtime Verification of Linearizability

We consider a standard concurrent shared memory system (e.g. [3]) with $n \geq 2$ *asynchronous* processes, $p_1, \ldots, p_n$, which may crash at any time in any execution, except for at least one that is guaranteed to remain operational. Processes communicate by invoking *atomic* operations on shared memory locations, such as *read/write*, *Compare&Set*, or *Fetch&Add*. Each process p_i is associated with a local state machine, A_i, and has an *index* i. Shared objects are denoted with uppercase letters (O), while local variables are denoted with lowercase letters subscripted by the process index (e.g., op_i, τ_i by process p_i).

These processes form a distributed algorithm $\mathcal{A}$, which is an *implementation* of a sequential object O (e.g. a queue, a lock, a snapshot, etc). The terms *algorithm* and *implementation* are used interchangeably.

An *execution* of an implementation $\mathcal{A}$ is a sequence of steps as specified by $\mathcal{A}$, combined with invocations and responses to operations, adhering to *well-formedness* properties [3]. Well-formedness ensures that a process begins a new operation only after its previous operation has received a response. *Correct* processes and *complete* and *pending* operations in an execution are defined in the usual way, as well as the *step complexity* of an implementation.

2.1. Linearizability

For an execution E of $\mathcal{A}$, $E|p_i$ denotes the sequence of invocations and responses of p_i in E. Executions E and F are *equivalent* if $\forall p_i$, $E|p_i = F|p_i$. For operations op and op' of E, op *precedes* op', denoted $op \prec_E op'$, if and only if the response of op appears before the invocation of op'. The operations are *concurrent*, denoted $op \|_E op'$, if neither $op \prec_E op'$ nor $op' \prec_E op$.

A finite execution E of $\mathcal{A}$ is *linearizable* [3] if it is possible to obtain an execution F by appending responses to some pending operations of E, and by removing the remaining pending operations, such that there exists a sequential execution S of O with F and S being equivalent and $\prec_F \subseteq \prec_S$. We say that $\mathcal{A}$ is *linearizable* if each of its finite executions are linearizable. We let the predicate $\mathsf{LIN}(E)$ to be true if and only if E is linearizable.

An execution F is *quasi-equivalent* to an execution E if for each p_i, $E|p_i = F|p_i \cdot x$, where x is the empty string or a single response. Thus, if E and F are equivalent, one is quasi-equivalent to the other, but not necessarily vice versa.

Lemma 2.1 (Quasi-equivalence and linearizability)**.** *Let E be any linearizable execution and F be any execution that is quasi-equivalent to E such that $\prec_F \subseteq \prec_E$. Then, F is linearizable.*

Proof. It follows from the definition of quasi-equivalence between executions that one can append zero or more responses to F, resulting in an execution G, such that G and E are equivalent and $\prec_G = \prec_F$. Hence, $\prec_G \subseteq \prec_E$, as by assumption $\prec_F \subseteq \prec_E$. Since E is linearizable, the responses that are appended to E and the pending operations that are removed from E (in order to linearize E) can be appended to and removed from G, resulting in an execution H such that there is sequential execution S equivalent to H with $\prec_H \subseteq \prec_S$. Thus, F is linearizable. □

Progress conditions. $\mathcal{A}$ is *wait-free* if in every infinite execution, every correct process completes infinitely many operations [44], and it is *lock-free* if in every infinite execution, infinitely many operations are complete [3]. The notions of wait-freedom and lock-freedom naturally extend to specific operations or fragments of pseudocode.

2.2. The Collect Object

The *collect* object [14] is a useful abstraction frequently used in shared memory algorithms. It can be seen as a shared array with n entries. Each process p_i can write v in the i-th entry using the $write(i, v)$ operation; the processes can read the whole memory using the $collect()$ operation, which is *not atomic* (hence the object cannot be specified by a sequential object) and hence for each entry it returns either the last written value in it or one of the values of the writes in that entry that are concurrent with the collect. Thus, an invocation to $collect()$ spans an *interval of time*, representing the time it needs to collect the values in the entries of the shared array. For instance, in Figure 1 the collect operations by processes p_3 and p_4 are concurrent and return different values (a or b) depending on the specific time r_1 and r_2 at which each process reads registers 1 and 2, respectively. This behavior occurs despite a being written prior to b. However, the final collect operation by p_3 returns all values, as both writes precede it.

The collect object can be easily and optimally wait-free implemented using a shared array M. When p_i invokes $write(i, v)$, it writes v in $M[i]$, and when it invokes $collect()$ it reads one by one, in some arbitrary order, the entries of M and returns the vector of values it obtains. We consider a collect implementation in Algorithms 1 and 3.

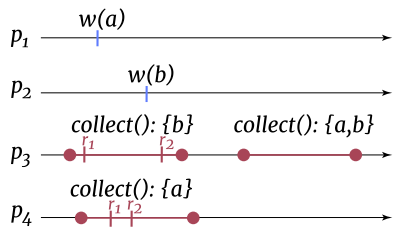


Figure 1: Example of 4-process execution of a collect implementation

2.3. Statement of the problem

To runtime verify the linearizability of any algorithm $\mathcal{A}$, we present two distributed algorithms: the Verifier $\mathcal{V}^+$ and the algorithm $\mathcal{A}^+$ (see Figure 2). The processes execute the Verifier $\mathcal{V}^+$, in which $\mathcal{A}$ its first transform into a wrapper $\mathcal{A}^+$, which it is a distributed algorithm that returns responses to invocations

of $\mathcal{A}$ and information about $\mathcal{A}^+$'s current execution, then the processes share and collect the Wrapper $\mathcal{A}^+$'s information to determine whether its linearizable.

The processes executing the verifier $\mathcal{V}^+$ interact with $\mathcal{A}$ though $\mathcal{A}^+$. We assume $\mathcal{A}$ is a black-box, meaning processes have no access to its internal specification or code. Thus, in any execution (interaction) with $\mathcal{A}$, the execution of $\mathcal{A}$ only contains invocation and responses, and not the internal computations. Furthermore, since $\mathcal{A}$ is a black-box, no verifier can be specifically designed for $\mathcal{A}$, it must work for any possible linearizable implementation $\mathcal{A}' \neq \mathcal{A}$. Consequently, any algorithm $\mathcal{A}'$ can be transformed into a wrapper $\mathcal{A}^{+\prime}$. The processes can invoke any high-level operation of $\mathcal{A}$. In [12] this is denoted as Apply(op_i), but here we denote it as the $inv(op_i)$, the invocation of op_i.

The Verifier $\mathcal{V}^+$ satisfies the requirements for a relaxed definition *Distributed Indirect and Predictive Runtime Verification*, meaning the problem of distributed indirect and predictive runtime verification is solvable for any sequential object O, the definition is the following:

Definition 2.2 (Distributed Indirect and Predictive Runtime Verification)**.** A wait-free verifier $\mathcal{V}_O$ *distributed indirectly and predictive runtime verifies* linearizability of an object O if, in every infinite execution E of $\mathcal{V}_O$ with an arbitrary input implementation $\mathcal{A}^+$, the following conditions are satisfied:

⋄ Strong soundness: If for every finite prefix E' of E, then $\mathsf{LIN}(E'|\mathcal{A}^+)$, either no process reports ERROR or a process reports ERROR together with a witness.

⋄ Completeness. If E has a finite prefix E' such that $\neg\mathsf{LIN}(E'|\mathcal{A}^+)$, at least one process reports ERROR together with a witness.

Linearizability of O is *indirectly predictive distributed runtime verifiable* if there is a wait-free verifier that distributed runtime verifies linearizability of O.

Broadly speaking, processes achieve verification through the following steps. An overview is illustrated in Figure 3, where a process executing the Verifier $\mathcal{V}^+$ invokes operations of the Wrapper $\mathcal{A}^+$, which in turn invokes operations of the target algorithm $\mathcal{A}$.

For a process p_i executing the Verifier $\mathcal{V}^+$ (see Algorithm 3):

1) Non-deterministically select an operation op_i of $\mathcal{A}$,

2) invoke an op_i through $\mathcal{A}^+$,

3) add op_i in its i-entry of the collect object (M),

4) obtain all operations written in M through a collect operation,

5) construct a sketch of $\mathcal{A}^+$'s execution from the collected operations (via Algorithm 2), and finally

6) decide whether the sketch is linearizable and return a witness of it (we left this step implicity).

For a process p_i executing the Wrapper $\mathcal{A}^+$ (see Algorithm 1):

1) Announce the invocation of the operation $inv(op_i)$ in its i-entry of the collect object (N),

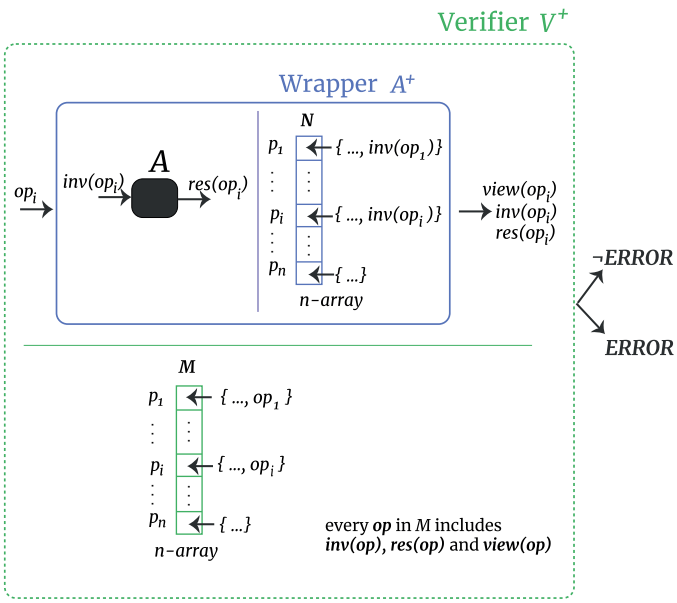


Figure 2: Representation of the Wrapper $\mathcal{A}^+$ and the Verifier $\mathcal{V}^+$

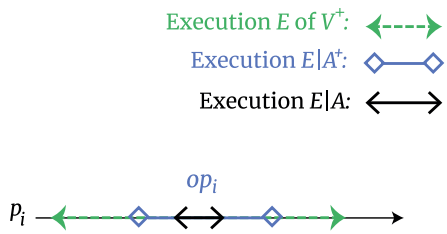


Figure 3: An execution of the Verifier $\mathcal{V}^+$

2) invoke op_i on $\mathcal{A}$ to obtain the response $res(op_i)$,

3) obtain the $view(op_i)$: all invocations written in N through a collect operation, and finally

4) return $res(op_i)$ together with $view(op_i)$.

Assuming no crash failures occur, all executions (or interactions) of the verifier may be infinite. This assumption, however, does not preclude $\mathcal{V}^+$ from being crash-tolerant, as the system is asynchronous and linearizability is verified over finite prefixes of these executions.

Algorithm $\mathcal{A}$ can implement progress conditions such as *wait-free* or *lock-free*, and the Wrapper $\mathcal{A}^+$ and the Verifier $\mathcal{V}^+$ preserve the progress condition of $\mathcal{A}$ (if locks were allowed, the resulting system that integrates $\mathcal{A}$ and $\mathcal{V}$ loses the progress properties of $\mathcal{A}$, i.e., lock-freedom or wait-freedom).

Remark. The term *predictive indirect* used in Definition 2.2 was recently clarified in [36]. It was originally introduced as *strong indirected* in [12]. However, within a decidability setting, the term predictive seems more appropriate, as it highlights the relaxed nature of the definition. Due to the strong soundness guarantee, we can predict that if $\mathcal{A}$ is incorrect, then the wrapper $\mathcal{A}^+$ will also be incorrect.

For common sequential objects such as queue, stack and consensus, there is no wait-free verifier that is sound and complete [12], namely, it is not true that processes report ERROR if

and only if the current execution of $\mathcal{A}$ is not linearizable. This is the reason of why in an execution E of $\mathcal{V}^+$ we verify the execution $E|\mathcal{A}^+$ (Wrapper $\mathcal{A}^+$) and not the execution $E|\mathcal{A}$ (see Figure 3). For every execution E, the execution $E|\mathcal{A}^+$ denotes the sequence of invocation and responses of $\mathcal{A}^+$. However, it is possible to "wrap" *any* $\mathcal{A}$ in an algorithm $\mathcal{A}^+$ that outputs responses to operations, together with information of its current execution, a "sketch" of it, such that the stronger problem is solvable. The relation between $\mathcal{A}$ and $\mathcal{A}^+$ it is: $\neg\mathsf{LIN}(\mathcal{A}^+) \implies \neg\mathsf{LIN}(\mathcal{A})$, and $\mathcal{A}^+$ can fix some executions by fulfilling the Strong soundness.

This model captures the execution interval of each operation, what we refer to as *interval detection*. This is reflected in the steps where an operation's invocation $inv(op_i)$ is recorded, and its corresponding view $view(op_i)$ and response $res(op_i)$ are returned by the wrapper and subsequently stored in the array M by the verifier $\mathcal{V}^+$.

3. Wrapper $\mathcal{A}^+$

In this section, we introduce the Wrapper $\mathcal{A}^+$, a wait-free, concurrent algorithm that is linearizable with respect to an object O. Let $\mathcal{A}$ be any implementation of O that is presumed to be linearizable. The Wrapper $\mathcal{A}^+$ invokes operations of $\mathcal{A}$ and returns both the responses from $\mathcal{A}$ and additional information to construct a "sketch" of its execution.

To show that $\mathcal{A}^+$'s sketch is not an exact representation of its execution but is sufficient to runtime verify $\mathcal{A}^+$, we define a special class of executions called *tight executions*.

A *tight execution* represents the execution as perceived by the processes through write and collect operations on the shared array. These executions are particularly useful because they precisely match the sketches constructed by the processes executing the Verifier $\mathcal{V}^+$ through the Algorithm 2 in the next section. Moreover, if $\mathcal{A}^+$'s execution is incorrect, then the corresponding tight execution is also incorrect, and consequently, the sketch will describe an incorrect execution.

Algorithm 1 The wrapper $\mathcal{A}^+$. Pseudocode of process p_i.

Shared Variables:
  N = Wait-free collect object with each entry initialized to $\emptyset$
Local Persistent Variable:
  $uset_i$ = an unbounded set initialized to $\emptyset$
Operation Apply(op_i) **is**
1: $uset_i \leftarrow uset_i \cup \{inv(op_i)\}$
2: N.write(i, $uset_i$)
3: Invoke operation, inv(op_i), of $\mathcal{A}$
4: $res(op_i) \leftarrow$ response from operation Apply(op_i) of $\mathcal{A}$
5: $c_i \leftarrow N$.collect()
6: $view(op_i) \leftarrow \bigcup_{k\in\{1,2,\ldots,n\}} c_i[k]$
7: **return** $res(op_i), view(op_i)$
  end Apply

The implementation $\mathcal{A}^+$ is presented in Algorithm 1. As anticipated, the processes use $\mathcal{A}$ to obtain responses to operations (Lines 3 and 4). To generate a sketch of $\mathcal{A}^+$'s current execution, each process p_i first announces its invocation, $inv(op_i)$, in its corresponding entry of N (Lines 1 and 2). Then, it collects the sets stored in N (Line 5) and finally returns the response $res(op_i)$ and the union of all collected sets (Lines 6 and 7), denoted as $view(op_i)$. The set $view(op_i)$ represents the *view* of operation op_i. Intuitively, $view(op_i)$ is the (unordered) set of invocations, $inv(op)$, corresponding to all completed or pending operations op that are concurrent with or precede op_i. Consequently, any completed operation op executed by any process p in the wrapper $\mathcal{A}^+$ consists of three components: an invocation event $inv(op)$, a response event $res(op)$, and a view $view(op)$.

In every execution of $\mathcal{A}^+$, each operation has an associated "inner" operation corresponding to the invocation and response from $\mathcal{A}$. For any execution E of $\mathcal{A}^+$, we define $E|\mathcal{A}$ as the subsequence containing only the invocations and responses of $\mathcal{A}$, representing its execution within E.

$\mathcal{A}^+$ assumes entries of N can store sets of unbounded size. This unrealistic assumption can be eliminated with linked lists: each p_i stores in the shared memory a double linked list that only p_i can modify but anyone can read. In this way, in Line 2, p_i writes in N a pointer to its most recent cell in its list.

Analyzing $\mathcal{A}^+$. Due to the optimal collect implementation in Section 2:

Lemma 3.1 (Step complexity of $\mathcal{A}^+$)**.** *In $\mathcal{A}^+$, every operation op takes the number of steps $\mathcal{A}$ takes for the same operation plus n reads and* 1 *write.*

$\mathcal{A}^+$ preserves the progress and correctness properties of $\mathcal{A}$. While progress is due to the fact that N is wait-free, linearizability follows from Lemma 2.1 and the fact that any execution E of $\mathcal{A}^+$ is quasi-equivalent to $E|\mathcal{A}$ and $\prec_E \subseteq \prec_{E|\mathcal{A}}$.

Lemma 3.2 (Progress and correctness of $\mathcal{A}^+$)**.** *$\mathcal{A}^+$ is wait-free (resp. lock-free) if and only if $\mathcal{A}$ is wait-free (resp. lock-free). Moreover, $\mathcal{A}^+$ is linearizable (disregarding views) if and only if $\mathcal{A}$ is linearizable.*

Proof. The proof is basically the same as proof of Lemma 7.2 in [12]. For completeness, we present a proof here.

For progress, since N is wait-free, the code in Lines 1, 2, 5 and 6 is wait-free. This directly implies that both $\mathcal{A}^+$ and $\mathcal{A}$ have the same progress properties.

For linearizability, consider any finite execution E of $\mathcal{A}^+$. First suppose that $\mathcal{A}$ is linearizable. Let us assume, wlog, that E has no pending operation that does not executes Line 3. Observe that: 1) E is quasi-equivalent to $E|\mathcal{A}$, as an operation that already has a response from $\mathcal{A}$ might be a pending operation of $\mathcal{A}^+$, due to asynchrony; (2) the specification of $\mathcal{A}^+$ directly implies that if $op \prec_E op'$ then $op \prec_{E|\mathcal{A}} op'$, and hence $\prec_E \subseteq \prec_{E|\mathcal{A}}$. By assumption, $E|\mathcal{A}$ is linearizable, and thus Lemma 2.1 gives that E is linearizable.

For the other direction, let us suppose that $\mathcal{A}^+$ is linearizable. Let us assume, wlog, that 1) every operation that is complete in $E|\mathcal{A}$, it is is complete in E too, and 2) E has no pending operation that does not executes Line 3. Note that E and $E|\mathcal{A}$ are equivalent. Due to asynchrony, there is an execution of E' of $\mathcal{A}^+$, equivalent to E (possibly with distinct views though), such that, for every operation, its invocation and steps in Lines 1 and 2 appear right before its invocation to $\mathcal{A}$ in Line 3, and its steps in Lines 5 and 6, and its response appear right after its

response from $\mathcal{A}$ in Line 4. We have that E' and $E|\mathcal{A}$ are equivalent. Finally, $E|\mathcal{A}$ is linearizable because E' is linearizable, by assumption.

□

We now define a subset of $\mathcal{A}^+$ executions such that, if detected at runtime, they enable the runtime verification of linearizability.

Tight executions. Consider an operation op_i of p_i in an execution E of $\mathcal{A}^+$. During its collect in Line 5 Algorithm 1, p_i performs up to n reads (fewer if op_i is pending). Let e be any of these reads, and $N[j]$ be the entry it reads. A read e is called *fresh* if either e reads a non-empty set and op_i is the first operation of p_i, or the set e reads from $N[j]$ differs from the one read from $N[j]$ by p_i in its previous collect operation. The *latest* fresh read (of op_i) is the fresh read that appears latest in E, and the *tail* of the collect consists of all reads that follow this latest fresh read. Intuitively, the latest fresh read marks the last step where op_i acquires new information about the ongoing execution. Since p_i writes $inv(op_i)$ to $N[i]$ before performing its collect, the latest fresh read is always well-defined.

An execution T of $\mathcal{A}^+$ is *tight* if for every operation: 1) if pending, it has its write in Line 2 but no read step of its collect in Line 5 and the invocation written in Line 2 appears in the collect of a complete operation; 2) its invocation and steps in Line 1 appear all right before its write; 3) if complete, the tail of its collect (if any) and its response *appear all right*[1] after its latest fresh read.

In summary, in a tight execution, the beginning and end of a complete operation correspond precisely to its invocation write and its latest fresh read.

Any execution E of $\mathcal{A}^+$ can be transformed into its *associated* tight execution $T(E)$ through the following modifications: 1) remove any pending operation that either has no write in Line 2 or whose invocation (written in Line 2) does not appear in the collect of a complete operation; 2) for every operation with a pending collect (i.e. with $< n$ reads), remove all these reads; then, for every remaining operation: 3) move forward, immediately before its write step, its invocation and its local steps in Line 1; 4) move backward, immediately before its latest fresh read, the tail of its collect (if any); and if the operation is complete, move backward the local steps in Line 6 and its response, otherwise insert at that place the corresponding local steps in Line 6 and response. Intuitively, $T(E)$ represents the execution the processes perceive through their views in E.

Figure 4 depicts a 3-process execution E of $\mathcal{A}^+$ and its associated tight execution $T(E)$. The latest fresh read of op_1 is r_2, as it reads from $N[2]$ the set containing the invocation $inv(op_2)$ written by p_2 in op_2 and r_3 reads $\emptyset$ from $N[3]$. In $T(E)$, op_1 spans the interval that goes from its write to r_2. Note that op_1' is not in $T(E)$ as its pair written in Line 2 does not appear in the collect of a complete operation. Also note that op_2 and op_3 are concurrent in E, but op_2 precedes op_3 in $T(E)$.

[1] The response event happens immediately, e.g., the tight execution in Figure 4.

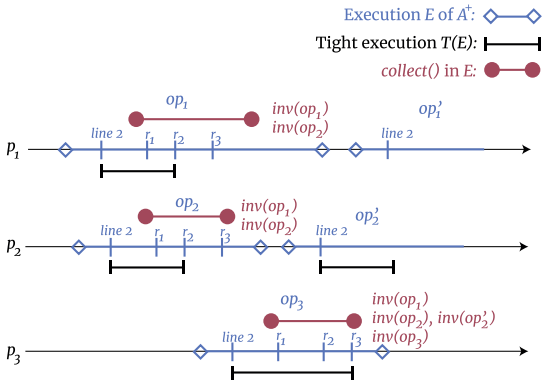


Figure 4: A tight execution of $\mathcal{A}^+$.

Lemma 3.3 ($T(E)$ is an execution of $\mathcal{A}^+$)**.** *For every execution E of $\mathcal{A}^+$, $T(E)$ is an execution of $\mathcal{A}^+$.*

Proof of Lemma 3.3. To prove that $T(E)$ is an execution of $\mathcal{A}^+$, observe that $T(E)$ only removes from E operations that do not write in N, and read steps of some (pending) operations. Now, in $T(E)$, the positions of write steps are not modified, only *local* steps are move forward or backward, as well as the tail of collects which moved backwards. Moving tails of collects is not a problem because the previous operation of the corresponding process read the same.

□

In the following Lemma, due to the order of operations in Algorithm 1, all preceding operations in any execution of $\mathcal{A}^+$ are also preceding in its tight execution. This it is enough to prove Corollary, as linearizability its strict with the order of preceding operations.

Lemma 3.4 (E and $T(E)$)**.** *Let E be an execution of $\mathcal{A}^+$. For any two operations op_i and op_j of E, complete or pending, such that op_i and op_j appear in $T(E)$:*

$$op_i \prec_E op_j \implies op_i \prec_{T(E)} op_j$$

$$op_i \parallel_E op_j \wedge op_i \prec_{T(E)} op_j \implies op_i \prec_{E|\mathcal{A}} op_j$$

Proof. Let op_i and op_j be two operations in E and in $T(E)$ by processes p_i and p_j, respectively.

If $op_i \prec_E op_j$ then, in E, the Write step in Line 2 of $inv(op_j)$ happens after the collect operation of p_i (Line 5 in Algorithm 2), then p_i does not see $inv(op_j)$. This remains true in $T(E)$, by definition of $T(E)$. Therefore, $op_i \prec_{T(E)} op_j$.

Now, if $op_i \parallel_E op_j$ and $op_i \prec_{T(E)} op_j$, then, in E and in $T(E)$, the collect operation (Line 5 in Algorithm 2) of process p_i happens before or concurrently to the write operation of the invocation (Line 2 in Algorithm 2) of p_j. Therefore, in E and in $T(E)$, the Line 3 by p_i happens before its Line 5, and the Line 3 by p_j succeeds its the Line 2. Thus, $op_i \prec_{E|\mathcal{A}} op_j$. □

Corollary 3.5. *Let E be an execution of $\mathcal{A}^+$. Then,*

$$\neg\mathsf{LIN}(E) \implies \neg\mathsf{LIN}(T(E)).$$

Proof. Let op_i and op_j be two operations in E and in $T(E)$ by processes p_i and p_j, respectively. By Lemma 3.4, if $op_i \prec_{T(E)} op_j$ then $op_i \prec_E op_j$ or $op_i \parallel_E op_j$, and if $op_i \parallel_{T(E)} op_j$ then $op_i \parallel_E op_j$.

By construction, some operations that are pending in E are completed in $T(E)$ and some pending operations in E do not appear in $T(E)$. Hence, for every process p_i, either $T(E)|p_i = E|p_i$, $T(E)|p_i = E|p_i \cdot res$ or $T(E)|p_i \cdot inv = E|p_i$, where res is the response of the last operation of p_i in $T(E)$ and inv is the invocation of the last pending operation of p_i in E.

If for every process p_i in $T(E)$, $T(E)|p_i = E|p_i$ or $T(E)|p_i = E|p_i \cdot res$, then $\prec_E \subseteq \prec_{T(E)}$, by Lemma 3.4. By Lemma 2.1, $\mathsf{LIN}(T(E)) \implies \mathsf{LIN}(E)$.

If for some process p_i in $T(E)$, $T(E)|p_i \cdot inv = E|p_i$, then inv is pending in E because does not have its collect operation. Moreover, inv is not in $T(E)$ because happens after every operation in $T(E)$, by construction. If $\mathsf{LIN}(T(E))$ then exists sequential execution S of O with $\prec_{T(E)'} \subseteq \prec_S$, such that $T(E)'$ is obtained by appending responses to some pending operations and removing the remaining pending operations. Hence, $\mathsf{LIN}(E)$, because exists sequential execution S of O with $\prec_{E'} \subseteq \prec_S$, where E' is an execution obtained by appending responses to some pending operations and removing the remaining pending operations as inv, thus $T(E)'|p_i = E'|p_i$ or $T(E)'|p_i = E'|p_i \cdot res$. □

The results above show that, if somehow the tight execution $T(E)$ of the current execution E can be detected in a verifier, linearizability can be runtime verified. For completeness, $\neg\mathsf{LIN}(E) \implies \neg\mathsf{LIN}(T(E))$, by Corollary 3.5, and hence the processes can report ERROR. For the case of strong soundness, we have two cases: 1) if $\mathsf{LIN}(T(E))$ then the processes do not report ERROR; 2) but if $\neg\mathsf{LIN}(T(E))$, then it must be the case that there is a pair of concurrent operations of E that one of them precedes the other in $T(E)$ (as in the second case of Lemma 3.4), i.e., $T(E)$ does not accurately capture E, it is only a "sketch" of E. However, $T(E)$ is an execution of $\mathcal{A}^+$, by Lemma 3.3, hence the processes have "discovered" that, although the current execution is linearizable, $\mathcal{A}^+$ is not linearizable (and thus $\mathcal{A}$ is not linearizable either, by Lemma 3.2).

4. Equivalence between Views and Tight Executions: Sketches

In this section, we present the algorithm $X(W_E)$, which constructs the current sketch of $\mathcal{A}^+$'s execution using the set of views and responses returned by $\mathcal{A}^+$. The primary result of this section is to demonstrate that the obtained sketch is linearizable if and only if its corresponding tight execution is linearizable.

Specifically, the processes executing the Verifier $\mathcal{V}^+$ in the subsequent section share its own operations returned by $\mathcal{A}^+$. We denote as W_E the set of all operations returned by the wrapper $\mathcal{A}^+$. Every operation op_i returned by $\mathcal{A}^+$ includes its invocation: $inv(op_i)$, response: $res(op_i)$ and view: $view(op_i)$.

The algorithm described here transforms the set of all returned operations, W_E, into a well-formed execution X_E. This transformation is a key technical contribution of our work. Notably, the views obtained by non-atomic reads of the shared array encode tight executions, despite the minimal guarantees provided by the collect operation.

Finally, we establish that the order of operations in X_E is equivalent to the order in its tight execution. Consequently, by Corollary 3.5, we prove that $\mathcal{A}^+$ can be verified through the set of responses shared by the processes.

The main challenge of using collects instead of snapshots [12] lies in constructing the sketch of $\mathcal{A}^+$'s execution, namely, the well-formed history X_E. This difficulty is formally captured in Lemma 4.1. A key advantage of snapshots is that their atomicity ensures that all returned views are comparable, meaning they can always be ordered by containment in ascending order (as shown in [12, 15]). In contrast, collects only guarantee an order by containment when the operations are not concurrent, as stated in the second item of Lemma 4.1. This limitation complicates the reconstruction of a well-formed execution history.

For the rest of the section, fix an arbitrary finite tight execution E of $\mathcal{A}^+$.

Lemma 4.1 (Properties of Views)**.** *Consider the views $view(op_i)$ and $view(op_j)$ in the responses of complete operations op_i and op_j of E by processes p_i and p_j (possibly with $p_i = p_j$). Then:*

1) Self-inclusion: $inv(op_i) \in view(op_i)$.

2) Containment in precedence: if $op_i \prec_E op_j$, then $view(op_i) \subset view(op_j)$.

3) Non-empty intersection: if $op_i \parallel_E op_j$, then $view(op_i) \cap view(op_j) \neq \emptyset$.

Proof. By the specification of $\mathcal{A}^+$, in operation op_i, p_i writes its invocation $inv(op_i)$ before its collect operation. Hence, its collect should read $inv(op_i)$, and consequently $inv(op_i) \in view(op_i)$.

Now, if $op_i \prec_E op_j$ then the collect operation op_i happens before the write of the invocation pair of op_j, and hence $inv(op_j) \notin view(op_i)$. Moreover, by the semantics of the collect operation, the collect of op_j justifying returns all invocations preceding it, and thus $view(op_i) \subset view(op_j)$.

Finally, if $op_i \parallel_E op_j$ then can happen two cases:

1) Line 2 of both operations happens before both collect operations in Line 5, therefore $inv(op_i), inv(op_j) \in view(op_i), view(op_j)$ and $view(op_i) \cap view(op_j) = \{inv(op_i), inv(op_j)\}$.

2) Line 2 of op_e is concurrent to the collect operation in Line 5 of op_f, therefore Line 2 of op_f happens before the Line 5 of op_e and by Containment in precedence property, $view(op_i) \cap view(op_j) = \{inv(op_j)\}$.

□

Lemma 4.1 suggests that it is possible to construct the well-formed execution X_E by analyzing the contention and intersection between views in E. This is essentially what the local Algorithm 2 achieves using those operations, as we show next.

Let W_E be the set containing every complete operation op returned by the wrapper $\mathcal{A}^+$, such that its invocation event its represented by $inv(op)$, the response event by $res(op)$, and its associated view $view(op)$, the set of invocations that are either preceding or concurrent with op.

Algorithm 2, executed locally by each process, takes W_E, analyzes the views it contains, and produces an execution X_E in two phases. In the first phase, Lines 1-8, the algorithm iterates over the complete operations in arbitrary order. In each iteration, it adds both the invocation and response of an operation op_i executed by process p_i somewhere into X_E. The position where the invocation $inv(op_i)$ is inserted (Line 6) follows Principle 4.2, which, roughly speaking, places it before all operations that precede op_i, according to the containment-based precedence relation described in Lemma 4.1, and before some responses that have not a precedence relation.

Likewise, the position where the response $res(op_i)$ is inserted (Line 7) follows Principle 4.3, and its intuitively, placed after some invocation events not ordered by containment with op_i, and before the responses of operations whose views do not include $inv(op_i)$ while still preceding all operations that follow op_i, following Lemma 4.1.

In the second phase, Lines 9-13, the algorithm handles pending invocations which are invocations for which there is neither a response, nor a returned view. It examines the responses in index-ascending order, and whenever it finds a response ($res(op_i)$) whose associated view ($view(op_i)$) has invocations ($inv(op)$) that are not yet in X_E, it inserts those missing invocationsl, in arbitrary order, immediately before the response $res(op_i)$. This ensures that the response is justified by its view.

Algorithm 2 From $\mathcal{A}^+$'s returned views (W_E) to the sketch (X_E)

Operation $X(\lambda_E)$ **is**:
1: $\varphi \leftarrow \emptyset$ ▹ Initialize the set of processed $\mathcal{A}^+$'s operations
2: $X_E \leftarrow \epsilon$ ▹ Initialize the sequence of invocations and responses
3: **do** ▹ Process all operations in W_E
4: $\gamma \leftarrow$ any operation in $W_E \setminus \varphi$
5: $\varphi \leftarrow \varphi \cup \{\gamma\}$
6: Add $inv(\gamma)$ to X_E following Principle 4.2
7: Add $res(\gamma)$ to X_E following Principle 4.3
8: **while** $\varphi \neq \lambda_E$
9: **for** $i \leftarrow 0$ to $|X_E| - 1$ ▹ Add pending invocations
10: **if** $X_E[i] \equiv res(op')$
11: $\beta := \{inv(op) \in view(op') \mid \forall k \in \{0, \ldots, |X_E| - 1\}, X_E[k] \neq inv(op)\}$
12: $X_E \leftarrow X_E[0 : i] \,\|\, \beta \,\|\, X_E[i : |X_E|]$
13: **return** X_E
end $X(\lambda_E)$

Principle 4.2 (Adding an invocation $inv(\gamma)$ to X_E)**.** Let γ be a complete operation selected in Line 4 of Algorithm 2, $inv(\gamma)$ is inserted before $X_E[i]$, such that for every event $e = X_E[j]$ where $i \leq j < |X_E|$, the following conditions hold:

1. If e is an $inv(op)$, then one of the following must hold:
   (a) $inv(\gamma) \in view(op)$ and $inv(op) \notin view(\gamma)$.
   (b) $inv(\gamma) \in view(op)$ and there exists a op' in W_E s.t: $inv(op') \in view(op)$, $inv(op) \notin view(op')$, $view(op') \subset view(op)$ and $inv(\gamma) \notin view(op') \quad \vee \quad view(op') \not\subset view(\gamma)$.
2. If e is a response pair $res(op)$, then one of the following must hold:
   (c) $inv(\gamma) \in view(op)$.
   (d) $view(op) \not\subset view(\gamma) \wedge view(\gamma) \not\subset view(op)$.

Principle 4.3 (Adding a response $res(\gamma)$ to X_E)**.** Let γ be a complete operation selected in Line 4 of Algorithm 2, $res(\gamma)$ is inserted before $X_E[i]$, such that for every event $e = X_E[j]$ where $i \leq j < |X_E|$, the following conditions hold:

1. If e is a $inv(op)$, then:
   (e) $inv(\gamma) \in view(op) \quad view(\gamma) \subset view(op)$.
2. If e is a $res(op)$, then one of the following must hold:
   (f) $view(\gamma) \subseteq view(op)$.
   (g) $view(op) \not\subset view(\gamma) \wedge view(\gamma) \not\subset view(op)$ and there exists a op' in W_E s.t: $inv(op') \in view(op) \oplus view(op') \wedge inv(\gamma) \in view(op') \wedge view(\gamma) \subset view(op')$.

For example, Figure 4 shows a tight execution E of $\mathcal{A}^+$, with the following operations and views:

$$\begin{aligned} W_E &= \{op_1, op_2, op_3\} \\ view(op_1) &= \{inv(op_1), inv(op_2)\} \\ view(op_2) &= \{inv(op_1), inv(op_2)\} \\ view(op_3) &= \{inv(op_1), inv(op_2), inv(op_2'), inv(op_3)\} \end{aligned}$$

Algorithm 2 produces the following execution:

$$\begin{aligned} X_E = \{&inv(op_1), inv(op_2), res(op_2), res(op_1), \\ &inv(op_3), inv(op_2'), res(op_3)\}. \end{aligned}$$

Observe that the execution $T(E)$ depicted in Figure 4 is precisely captured by X_E. The invocation $inv(op_1')$ does not appear in X_E because it is not included in the view of any complete operation returned by the $\mathcal{A}^+$ wrapper, that is, it is not part of W_E.

Note that the specific order of certain invocations and responses in X_E is not relevant, as reordering them does not affect the precedence or concurrency relations among operations. For example, the execution X_E', show below, can also be generated by Algorithm 2, and is equivalent to X_E with $\prec_{X_E} = \prec_{X_E'}$:

$$\begin{aligned} X_E' = \{&inv(op_2), inv(op_1), res(op_1), res(op_2), \\ &inv(op_3), inv(op_2'), res(op_3)\}. \end{aligned}$$

For the remainder of this section, we adopt the following notation: 1) For any operation op_i of X_E, we write $cmplt(op_i)$ to indicate that op_i is complete in X_E; that is, both its invocation and response appear in X_E.

2) For each complete operation op_i in X_E, issued by any process p_i, we assume $op_i \in W_E$. This means that the operation is returned by the $\mathcal{A}^+$ wrapper, and its associated triple $(inv(op_i), res(op_i), view(op_i))$, representing its invocation, response, and view, is known (Remark 4.4).

3) For any two views $view(op_i)$ and $view(op_j)$, we define $incomp(view(op_i), view(op_j))$ to hold if and only if the views are incomparable, that is: $view(op_i) \nsubseteq view(op_j) \wedge view(op_j) \nsubseteq view(op_i)$.

Remark 4.4. Let op_i be any operation in X_E by p_i, by Algorithm X_E in 2:

$$cmplt(op_i) \iff \exists\, inv(op_i), res(op_i), view \in W_E$$

The following lemma is the key to prove that X_E indeed captures E (Theorem 4.9). It describes the order of precedence and concurrency of operations based on containment and inclusion between views in W_E.

Lemma 4.5 (Relating X_E and W_E)**.** *Let op_i be a complete operation in X_E by process p_i, and op_j be a complete or pending operation in X_E by process p_j.*

1. $op_i \prec_{X_E} op_j \iff$

   $(cmplt(op_j) \wedge view(op_i) \subset view(op_j) \wedge inv(op_i) \in view(op_j)) \vee$

   $(\neg cmplt(op_b) \wedge inv(op_j) \notin view(op_i))$

2. $op_i \parallel_{X_E} op_j \iff$

   $(cmplt(op_j) \wedge inv(op_i) \in view(op_j) \wedge inv(op_j) \notin view(op_i) \wedge incomp(view(op_i), view(op_j))) \vee$

   $(cmplt(op_j) \wedge inv(op_i) \in view(op_j) \wedge inv(op_j) \in view(op_i) \wedge (incomp(view(op_i), view(op_j)) \vee view(op_i) \subset view(op_j))) \vee$

   $(\neg cmplt(op_j) \wedge inv(op_j) \in view(op_i))$

Proof. We show that the relation $\prec_{X_E}$ and the concurrency relation $\parallel_{X_E}$ are fully determined by the views associated with operations in X_E. This conclusion follows from Principles 4.2 and 4.3, the well-formedness of E, Lemma 4.1, and the behavior of Algorithm 2. To begin, we establish that Principles 4.2 and 4.3 hold between every pair of invocations, responses and invocation/response in Claims 4.6, 4.7 and 4.8.

In the following claims, we demonstrate that Principles 4.2 and 4.3 apply to every pair of events in X_E under the following setup. Let op_i and op_j be any two complete operations in X_E. Denote their invocation and response events as $X_E[i] = inv(op_i)$, $X_E[j] = inv(op_j)$, $X_E[i'] = res(op_i)$ and $X_E[j'] = res(op_j)$, with indices i, j, i', j' in the range $[0, |X_E - 1|]$ such that $i < j$, $i < i'$ and $j < j'$. The case $(i' < j')$ is depicted in Figures 5(a) and 5(c), where $view(op_i) \subset view(op_j)$. In Figure 5(a), the operations have a precedence relation with $(i' < j)$, while in Figure 5(c), the operations are concurrent with $(j < i')$. The case $(j' < i')$ is depicted in Figure 5(b), where $incomp(view(op_i), view(op_j))$.

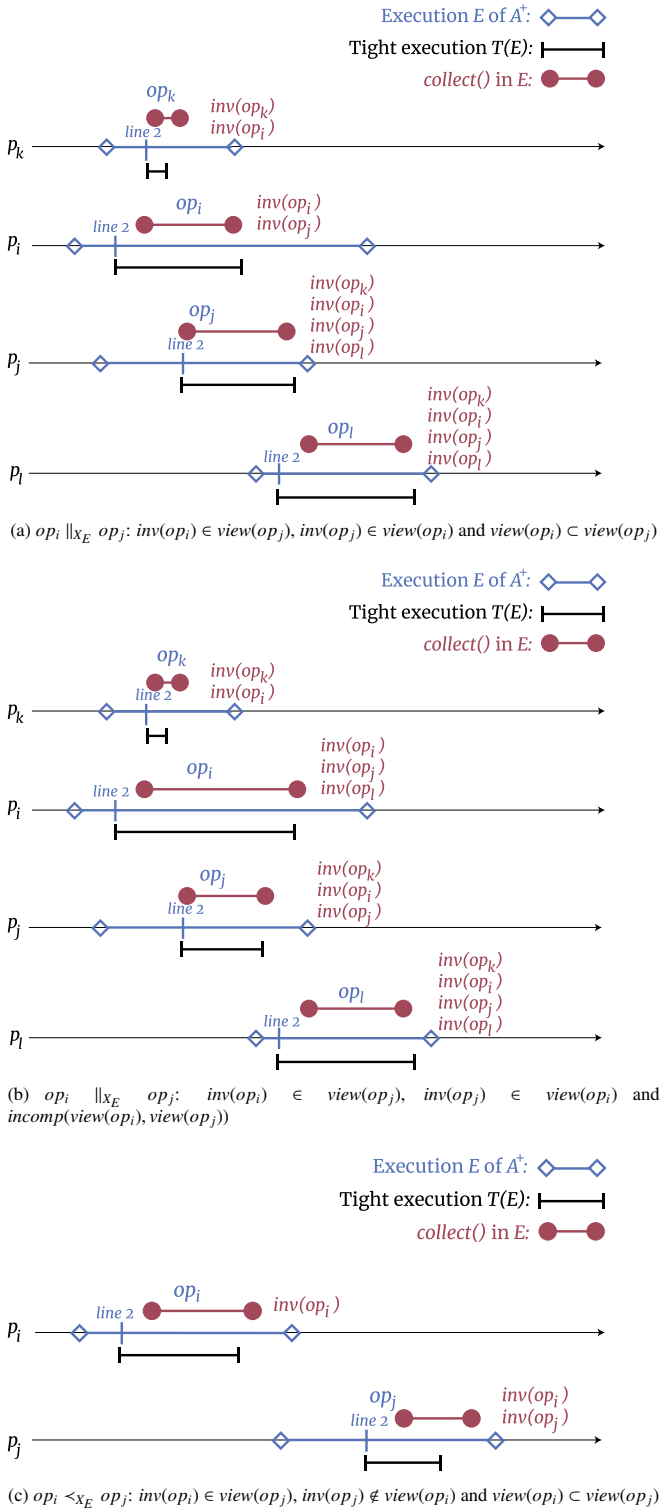


Figure 5: Representation of Principles 4.2 and 4.3

We first consider the ordering between invocations in X_E, as formalized in the following claim. The proof proceeds by contradiction: we assume that two invocations are related by Principle 4.2, but there exists a third invocation for which this principle does not hold. We show that this leads to a contradiction by analyzing all possible cases illustrated in Figure 5, covering

both concurrent and precedence relations between operations.

Claim 4.6. *For any pair of complete operations in X_E, op_i and op_j, Principle 4.2 hold between its invocations, $inv(op_i)$ and $inv(op_j)$.*

Proof. Suppose, by contradiction, that exists an invocation event $X_E[a] = inv(op_a)$, with $i < j < a$, for which Principle 4.2 holds between $inv(op_j)$ and $inv(op_a)$, but does not hold between $inv(op_i)$ and $inv(op_a)$. Then, we must have $a < i$ and $inv(op_a) \in view(op_i)$. By assumption, $inv(op_j) \in view(op_a)$, and due to the $X_E[j]/X_E[a]$ relation, we consider the following cases:

Case (i) : $inv(op_a) \notin view(op_j)$. This contradicts the scenario where $(i' < j')$ and $view(op_i) \subseteq view(op_j)$.

Case (ii) : there exists a op' such that $inv(op') \in view(op_a)$, $inv(op_a) \notin view(op')$, $view(op') \subset view(op_a)$, $view(op') \not\subset view(op_a)$. This contradicts the scenario where $(i' < j')$ and $view(op_i) \subset view(op_j)$, since $inv(op') \in view(op_a), view(op_i), view(op_j)$ (to maintain the relation between $X_E[a]/X_E[i]$).

Moreover, both cases also contradict the scenario $(j' < i')$ in which $incomp(view(op_i), view(op_j))$ holds. In this scenario, there exists an operation op'' in W_E such that $inv(op_j) \in view(op'')$, $view(op_j) \subset view(op'')$ and $inv(op'') \notin view(op_j)$, then $view(op_i) \oplus view(op_j) = \{inv(op_a), inv(op'')\}$ and due to the $X_E[i]/X_E[j]$ we obtain $view(op_a) \subset view(op_j)$ and $inv(op_a) \in view(op_j)$, leading to a contradiction.

Therefore the Principle 4.2 should fulfill between $X_E[i]$ and $X_E[a]$ if its hold with $X_E[j]$.

□

Next, we address the ordering between responses in X_E, as captured in the following claim. As in the previous case, the proof proceeds by contradiction: we assume two responses satisfy Principle 4.2, yet there exists a third response for which the principle fails. We derive a contradiction by examining all relevant scenarios shown in Figure 5, including both precedence and concurrency cases—covering incomparable views as well as containment relationships.

Claim 4.7. *For any pair of complete operations, op_i and op_j, Principle 4.3 hold between its response events, $res(op_i)$ and $res(op_j)$.*

Proof. Suppose, by contradiction, that exists a response event $X_E[b] = res(op)$, with $i' < j' < b$, where Principle 4.3 holds between $res(op_j)$ and $res(op_b)$, but does not hold between $res(op_i)$ and $X_E[b] = res(op_b)$. By the $X_E[i']/X_E[j']$ relation $view(op_i) \subseteq view(op_j)$ and by the $X_E[j']/X_E[b]$ relation $view(op_b) \not\subset view(op_j)$. Thus, we must have $b < i'$:

Case (i) : $view(op_b) \subseteq view(op_i)$. This contradicts $view(op_i) \subseteq view(op_j)$.

Case (ii) : $incomp(view(op_b), view(op_i))$ and there exists an operation op' in W_E such that $inv(op') \in view(op_b) \oplus view(op_i)$, $inv(op_b) \in view(op')$, $inv(op') \notin view(op_b)$ and $view(op_b) \subset view(op')$. It also contradicts, $view(op_i) \subset view(op_j)$, since we additionally have $view(op_j) \subseteq view(op_b)$ and $inv(op') \in view(op_i)$, leading to a contradiction.

Suppose, by contradiction, that $X_E[b] = res(op_b)$, with $j' < i' < b$, where Principle 4.3 holds between $X_E[i'] = (res(op_i))$ and $X_E[b] = res(op_b))$, but does not hold between $X_E[j'] = (res(op_j))$ and $X_E[b] = res(op_b))$. From this, we must have $b < j'$, $view(op_j) \not\subset view(op_b)$ and $\nexists\, op'$ in W_E such that $inv(op') \in view(op_b) \oplus view(op_j), view(op_b)$, $inv(op_j) \in view(op')$ and $view(op_j) \subset view(op')$. By the $X_E[i']/X_E[j']$ relation, $incomp(view(op_i), view(op_j))$ and there exists an operation op'' in W_E such that $inv(op'') \in view(op_i), view(op_i) \oplus view(op_j)$, $inv(op_j) \in view(op'')$, $view(op_j) \subset view(op'')$ and $inv(op'') \notin view(op_j)$. Furthermore, by the $X_E[i']/X_E[b]$ relation, we know that $view(op_b) \not\subset view(op_i)$, and one of the following cases:

Case (i): $view(op_i) \subseteq view(op_b)$. Ths contradicts $inv(op') \in view(op_b)$.

Case (ii): $incomp(view(op_i), view(op_b))$ and $\exists\, op$ such that exists $inv(op) \in view(op_b) \oplus view(op_i), view(op_b), inv(op_i) \in \lambda$ and $view(op_i) \subset \lambda$. This leads to a contradiction, as it implies $inv(op) \equiv inv(op')$, contradicting the assumed structure of $view(op_b)$.

□

Finally, we analyze the relation between the invocation of one operation and the response of another, as stated in the following claim. The proof, again by contradiction, shows that if Principles 4.2 and 4.3 fail to hold between such cross pairs, it leads to inconsistencies with the views, considering all concurrency and precedence scenarios illustrated in Figure 5.

Claim 4.8. *For any pair of complete operations, op_i and op_j, Principles 4.2 and 4.3 hold between it invocation/response events, $inv(op_i)$ and $res(op_j)$, and $inv(op_j)$ and $res(op_i)$.*

Proof. Suppose, by contradiction, that $X_E[a] = inv(op_a)$, with $a < i' < j'$, where Principle 4.2 holds between $X_E[a] = inv(op_a)$ and $X_E[i'] = res(op_i)$, but does not hold between $X_E[a] = inv(op_a)$ and $X_E[j'] = res(op_j)$. Since Principle 4.3 must hold between $X_E[j']$ and $X_E[a]$: $inv(op_j) \in view(op_a)$, $inv(op_a) \notin view(op_j)$ and $view(op_j) \subset view(op_a)$, By assumption $view(op_i) \subseteq view(op_j)$, this leads to a contradiction because $inv(op_a) \in view(op_i), view(op_j)$, implies $inv(op_a) \in view(op_j)$.

Suppose, by contradiction, that $X_E[a] = inv(op_a)$, with $a < j' < i'$, where Principle 4.2 holds between $X_E[a] = inv(op_a)$ and $X_E[j'] = res(op_j)$, but does not hold between $X_E[a] = inv(op_a)$ and $X_E[i'] = (res(op_i))$. By assumption, $incomp(view(op_i), view(op_j))$ and there exists an operation op'' in W_E such that $inv(op'') \in view(op_i), view(op_i) \oplus view(op_j)$, $inv(op_j) \in view(op'')$, $view(op_j) \subset view(op'')$ and $inv(op'') \notin view(op_j)$. By the $X_E[a]/X_E[j']$ relation, we have $inv(op_a) \in view(op_j)$ or $incomp(view(op_a), view(op_j))$. If Principle 4.3 holds between $X_E[i']$ and $X_E[a]$, then: $inv(op_i) \in view(op_a)$, $inv(op_a) \notin view(op_i)$ and $view(op_i) \subset view(op_a)$. This contradicts the fact that $inv(op'') \in view(op_i), view(op_i) \oplus view(op_j)$.

□

To complete the proof, consider any two complete operations op_i and op_j in X_E, by processes p_i and p_j respectively, with $i \neq j$. By the containment in precedence property in Lemma 4.1, the well-formedness property of processes, the fact that Principles 4.2 and 4.3 hold for every pair of invocations (Claim 4.6), responses (Claim 4.7) and invocations/responses (Claim 4.8), it follows that $op_i \prec_{X_E} op_j$. Moreover, by the non-empty intersection and self-inclusion properties in Lemma 4.1, we also have $op_a \parallel_{X_E} op_b$.

Now, consider the case where op_i is a complete operation and op_j is a pending operation in $X(W_E)$, by processes p_i and p_j, with $i \neq j$. Algorithm 2, specifically Lines 9-12, ensures that all pending invocations are added to X_E. Define indexes k, k', l within the range $[0, |X_E - 1|]$ such that $k < l$. Let $X_E[k] = inv(op_i)$, $X_E[k'] = res(op_i)$ and $X_E[l] = inv(op_j)$. If $k' < l$, the by Line 11, $inv(op_j) \notin view(op_i)$. By Claim 4.7, by the containment in precedence property in Lemma 4.1 and by well-formedness property of processes, it follows that $op_a \prec_{X_E} op_b$. If $l < k'$. By Line 11, $inv(op_j) \in view(op_i)$ or $\exists op$ such that $inv(op_j) \in view(op)$ and $incomp(view(op), view(op_i))$. By Claim 4.7, by the containment in precedence property in Lemma 4.1 and by well-formedness property of processes, it follows $op_a \parallel_{X_E} op_b$.

□

We can prove Theorem 4.9, the main result of this section. The proof consists in showing that, in E, if one operation precedes the other, their views satisfy one of the two clauses in the first item of Lemma 4.5, and, similarly, if two operations are concurrent, their views satisfy one of the clauses in the second item. The first item of Lemma 4.5 is crucial for verifying linearizability. Since linearizability imposes a strict order on preceding operations, this can be achieved through the *containment in precedence* property (Lemma 4.1) in the views.

Theorem 4.9 (Relating X_E and E). *For every tight execution E of $\mathcal{A}^+$, E and X_E are equivalent with $\prec_E = \prec_{X_E}$. Consequently,*

$$\mathsf{LIN}(E) \iff \mathsf{LIN}(X_E).$$

Proof. Let op_i and op_j be two complete operations in any tight execution E of $\mathcal{A}^+$ by processes p_i and p_j, respectively. Each operation has its invocation, response and view, *inv*, *res* and *view*, respectively (Remark 4.4).

Consider $op_i \prec_E op_j$, the Containment in precedence property in Lemma 4.1 implies $view(op_i) \subset view(op_j)$, then $inv(op_i) \in view(op_j)$ and $inv(op_j) \notin view(op_i)$, therefore by Lemma 4.5, $op_i \prec_{X_E} op_j$.

Consider $op_a \prec_{X_E} op_b$. By Lemma 4.5, $inv(op_i) \in view(op_j)$ and $inv(op_j) \notin view(op_j)$ and $view(op_i) \subset view(op_j)$, by the specification of $\mathcal{A}^+$, the write step of the invocation $inv(op_j)$ (Line 2 in Algorithm 2) happens after the collect operation of p_i (Line 5 in Algorithm 2), therefore for a tight execution definition, $op_i \prec_E op_j$.

Consider $op_i \parallel_E op_j$. By the specification of $\mathcal{A}^+$ the write step (Line 2) of both processes happens before both collect operation (Line 5), or the Write step (Line 2) of process p_i (resp. process p_j) happens concurrently the collect operation of process p_j, by the definition of a tight execution, the write step of process p_i is saw by p_j, hence $inv(op_i) \in view(op_j)$ and $inv(op_j) \in view(op_i)$, then $op_i \parallel_{X_E} op_j$.

Consider $op_i \parallel_{X_E} op_j$. By Lemma 4.5, *(i)* $inv(op_i) \in view(op_j)$ and $inv(op_j) \in view(op_i)$ or *(ii)* $inv(op_i) \in view(op_j)$ and $incomp(view(op_j), view(op_i))$. If (*i*), then the write step (Line 2) of both processes happens before or concurrently both collects in Line 5, therefore $op_i \parallel_E op_j$. If (*ii*), then the write step (Line 2) of process p_i happens before or concurrently the collect operation (Line 5) of process p_j, therefore $op_i \parallel_E op_j$.

Let op_i be a complete operation and op_j be a pending operation in any tight execution E of $\mathcal{A}^+$ by processes p_i and p_j, respectively. By Remark 4.4 op_i has invocation, response and view, and op_j has only invocation.

Consider $op_i \prec_E op_j$. By the specification of $\mathcal{A}^+$, the write step of the invocation $inv(op_j)$ (Line 2 in Algorithm 2) happens after the collect of p_i (Line 5 in Algorithm 2), hence $inv(op_j) \notin view(op_i)$ and by Lemma 4.5, $op_j \prec_{X_E} op_i$.

Consider $op_i \prec_{X_E} op_j$. By Lemma 4.5, $inv(op_j) \notin view(op_i)$, which implies that the write step of the invocation $inv(op_j)$ (Line 2 in Algorithm 2) happens before the collect of p_i (Line 5 in Algorithm 2), therefore for a tight execution definition, $op_j \prec_E op_i$.

Consider $op_j \parallel_E op_i$. By the specification of $\mathcal{A}^+$, the write step (Line 2) of process p_j happens concurrently or before the collect operation of process p_i, by the definition of a tight execution, the write step of process p_j is the collect of p_i, hence $inv(op_j) \in view(op_i)$ and by Lemma 4.5, $op_j \parallel_{X_E} op_i$.

Consider $op_j \parallel_{X_E} op_i$. By Lemma 4.5, $inv(op_j) \in view(op_i)$, hence the write step (Line 2) of p_j happens before or concurrently the collect operation of p_i in Line 5, therefore $op_j \parallel_E op_i$.

Therefore $\prec_{X_E} = \prec_E$ and an execution X_E is equivalent to a tight execution E, because for each p_i, $E|p_i = X_E|p_i$, as every complete operation in E is in X_E, by Remark 4.4 and by the definition of a tight execution; also every pending invocation in E is pending in X_E, because the pending invocation is in any collect of a complete operation and does not have its corresponding operation in W_E, by Remark 4.4. Hence, Lemma 2.1 implies $\mathsf{LIN}(X_E) \implies \mathsf{LIN}(E)$ and $\mathsf{LIN}(E) \implies \mathsf{LIN}(X_E)$.

□

5. The Verifier $\mathcal{V}^+$

In this section we present a verifier, $\mathcal{V}^+$, that runtime verifies linearizability of any algorithm $\mathcal{A}$ using the generic transformation in Section 3 to obtain $\mathcal{A}^+$.

The Verifier is detailed in Algorithm 3. Processes executing the Verifier $\mathcal{V}^+$ invoke operations of $\mathcal{A}^+$ and share their results in their respective entries of the shared array M. Each operation op_i shared in M includes its invocation $inv(op_i)$, response $res(op_i)$, and the associated view $view(op_i)$. This approach ensures that each process can access all operations previously written by other processes through a collect operation

on M, convert these tuples into a current execution X_E, and perform the verification procedure (which we leave implicit).

In $\mathcal{A}^+$, each process performs a collect to read all entries of M in a non-atomic manner. This differs from the model with atomic snapshots in [12], where more precise information about concurrency can be retained. As a result, some information about concurrent operations may be lost in $\mathcal{A}^+$. Similarly, the verifier $\mathcal{V}^+$ also operates with non-atomic collects and, consequently, may lose information about concurrent operations in the same way. In particular, while each process reads all operations written before its collect, it might miss operations that were returned by the wrapper but not yet written into M. That is, obtaining a response from $\mathcal{A}^+$ and writing that information into M are two distinct steps, and this separation can lead to inconsistencies in the collected data.

In each iteration of the while loop in Algorithm 3, process p_i invokes a non-deterministically chosen operation op_i of $\mathcal{A}^+$ (Lines 3 to 5), and writes op_i to $M[i]$ (Line 7), including $inv(op_i)$, $res(op_i)$, and $view(op_i)$. Then, p_i collects all operations in M (Line 8), and defines the set τ_i with all the operations in the collect (Line 9).

This collected set, τ_i, does not always match the set W_E defined in Section 4. Even though Theorem 4.9 suggests a seemingly straightforward approach to verify $\mathcal{A}^+$, the asynchronous behavior of processes and the non-atomic nature of collect operations, can lead to missing operations in τ_i. For instance, a process p_i might obtain the response to its operation from $\mathcal{A}^+$ (Line 5) but delay writing the corresponding operation op_i to its array entry in $M[i]$ before another process performs a collect operation (Line 8). Similarly, concurrent writes by multiple processes to M (Line 7) may overlap with a collect operation, causing certain operations to be absent from τ_i (Figure 6).

To account for this loss of information and avoid non-well-formed executions, we define a local function, $refine()$, which extracts a subset τ'_i of τ_i (Line 10). This refined set serves as input to Algorithm 2, enabling the construction of the current execution to verify.

As we will see, $X(\tau'_i)$ is indeed related to the actual execution of $\mathcal{A}^+$, and hence is the execution p_i tests against linearizability (Lines 11 and 12).

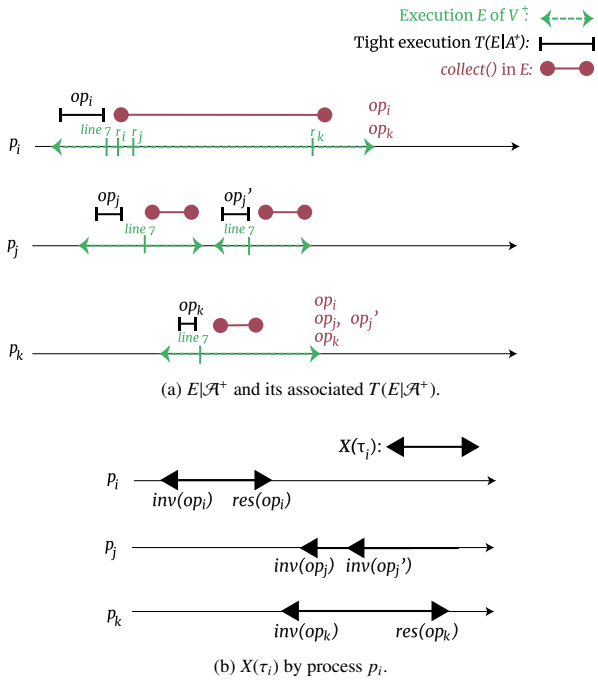
Algorithm 3 Verifier $\mathcal{V}^+$ for linearizability of any algorithm $\mathcal{A}^+$. Pseudocode of process p_i.

Shared Variables:
  M = Wait-free collect object with each entry initialized to $\emptyset$
Local Persistent Variable:
  res_i = an unbounded set initialized to $\emptyset$
Operation Verify($\mathcal{A}^+$) **is**
1: $res_i \leftarrow \emptyset$
2: **while** true **do**
3: $op_i \leftarrow$ non-deterministically chosen operation
4: Invoke operation Apply(op_i) of $\mathcal{A}^+$
5: Receive op_i from Apply(op_i) of $\mathcal{A}^+$
6: $res_i \leftarrow res_i \cup \{op_i\}$ ▹ op_i includes inv(op_i), res(op_i), and view(op_i)
7: M.write(i, res_i)
8: $c_i \leftarrow M$.collect()
9: $\tau_i \leftarrow \bigcup_{k\in\{1,2,\ldots,n\}} c_i[k]$
10: $\tau'_i \leftarrow refine(\tau_i)$
11: **if** $\neg$LIN($X(\tau'_i)$) **then**
12: **report**(ERROR, X(τ'_i))
  end Verify

Before defining function $refine()$, we motivate its necessity with an example. The main concern is that $X(\tau_i)$ may fail to represent a well-formed execution, which invalidates any attempt to test it for linearizability. Figure 6(a) ilustrates a 3-process execution E of $\mathcal{V}^+$, due to the non-atomicity and asynchrony of processes, the collect of operation op_i of p_i, "misses" both operations of p_j, op_j and op'_j, but it reads the operation op_k of p_k. As a result, the collected set τ_i includes only op_i and op_k, along with their invocations, responses, and associated views.

The problem arises from the fact that the view of op_k, namely $view(op_k)$, includes the invocations of the four operations ($inv(op_i)$, $inv(op_j)$, $inv(op'_j)$ and $inv(op_k)$), and hence τ_i encodes two pending operations of p_j, which violates well-formedness. Indeed, as shown in Figure 6(b), applying Algorithm 2 to τ_i produces a non-well-formed execution $X(\tau_i)$ in which process p_j appears to have two pending operations

This phenomenon is inherent to asynchrony: the set τ_i may be a strict subset of $W_{T(E|\mathcal{A}^+)}$, the set of all complete operations returned by $\mathcal{A}^+$ in $T(E|\mathcal{A}^+)$. Nonetheless, there always exists a subset $\tau'_i \subseteq \tau_i$ such that $X(\tau'_i)$ is well-formed and corresponds to a valid execution of $\mathcal{A}^+$. Intuitively, τ'_i is the maximal subset of τ_i in which each process has at most one pending operation. In the example from Figure 6(a), that subset contains only the operation of op_i of p_i.



(a) $E|\mathcal{A}^+$ and its associated $T(E|\mathcal{A}^+)$.

(b) $X(\tau_i)$ by process p_i.

Figure 6: Discrepancies between $X(\tau_1)$ and $T(E|\mathcal{A}^+)$.

Let E be any finite execution of $\mathcal{V}^+$, and let τ_i be the set obtained by process p_i in Line 9 in its last iteration of the while loop. Consider the set $W_{T(E|\mathcal{A}^+)}$ with the complete operations obtained from $\mathcal{A}^+$ in the tight execution $T(E|\mathcal{A}^+)$ associated to $E|\mathcal{A}^+$. Note that $\tau_i \subseteq W_{T(E|\mathcal{A}^+)}$. An invocation $inv(op)$ is τ_i-*pending* if: (1) $inv(op)$ appears in any $view(op_a)$ such that $op_a \in \tau_i$, and (2) there are no $res(op)$ or $view(op)$ such that $op \in \tau_i$. We define $refine(\tau_i)$ as the maximal subset of τ_i such

that for each of its operations, there are no pair of τ_i-pending invocations $inv(op'), inv(op'') \in view(op)$ such that $p' = p''$ and $op' \neq op''$. For instance, consider $\tau'_i = refine(\tau_i)$ in Figure 6(b). The execution $X(\tau'_i)$ corresponds to the same execution as $X(\tau_i)$ depicted in Figure 6(b), excluding the pending operation $inv(op'_j)$.

Lemma 5.1 ensures that the subset τ'_i constructed by a process p_i always includes at least its last own operation obtained from $\mathcal{A}^+$ and contains at most one pending operation for each other process.

Lemma 5.1 (Non-emptiness of τ'_i)**.** *Let E be any finite execution of $\mathcal{V}^+$, and let τ'_i be the set obtained by process p_i in Line 10 in its last iteration of the while loop. Then, τ'_i contains every operation op that appears in M right before the first read of p_i's collect in its last iteration of the while loop.*

Proof. By the specification of $\mathcal{V}^+$, at all times the $view(op)$ of an operation op in M has at most one pending invocation pair per process. Thus, when p_i starts its collect, all operations in M will end up in τ'_i, since, by the specification of collect, all of them appear in τ_i. □

We now argue that, despite asynchrony, $X(\tau'_i)$ indeed captures an execution of $\mathcal{A}^+$, possibly different from the current one, however $X(\tau'_i)$ it is only a subexecution of the current execution of $\mathcal{A}^+$. The proof consists in showing that there exists an execution F where some operations of the current execution are missing such that its tight execution $T(F|\mathcal{A}^+)$ is exactly $X(\tau'_i)$.

Lemma 5.2 ($X(\tau'_i)$ is an execution of $\mathcal{A}^+$)**.** *Let E be any finite execution of $\mathcal{V}^+$, and let τ'_i be the set obtained by process p_i in Line 10 in its last iteration of the while loop.*

There exist a finite execution F of $\mathcal{V}^+$ such that:

1) $T(F|\mathcal{A}^+)$ and $X(\tau'_i)$ are equivalent with $\prec_{X(\tau'_i)} = \prec_{T(F|\mathcal{A}^+)}$.

2) $F|\mathcal{A}^+$ is an execution of $\mathcal{A}^+$.

Proof. Let E be any finite execution of $\mathcal{V}^+$and let τ'_i be the set obtained by any process p_i in Line 9 in its last iteration of the while loop. Let $W_{T(E|\mathcal{A}^+)}$ be the set with the complete operations in the tight execution $T(E|\mathcal{A}^+)$ associated to $E|\mathcal{A}^+$. Let F be the execution obtained from E as follows: remove the steps in Lines 7 to 13 in $\mathcal{V}^+$ of all first operations of each process p_j that do not have its corresponding operation in τ'_i, then remove all the following operations that do not have its operation in τ'_i. Indeed, for any operation $op \in W_{T(E|\mathcal{A}^+)} \setminus \tau'_i$ and any operation $op' \in \tau'_i$, $op' \prec_{T(E|\mathcal{A}^+)} op$ or $op' \parallel_{T(E|\mathcal{A}^+)} op$.

We can transform $F|\mathcal{A}^+$ into a tight execution $T(F|\mathcal{A}^+)$ by *(1)* removing every pending operation with no write in Line 2 or with a write in Line 2 not read by any complete collect; *(2)* for every operation with a pending collect (i.e. with $< n$ reads), remove all these reads. Then, for every remaining operation: *(3)* move forward, right before its write step, its invocation and its local steps in Line 1; *(4)* move backward, right before its latest fresh read, the tail of its collect (if any); and if the operation is complete, move backward the local steps in Line 6 and its response, otherwise insert at that place the corresponding local steps in Line 6 and response. By construction, we have that $\tau'_i = W_{T(F|\mathcal{A}^+)}$, from which follows that $X(\tau'_i)$ and $X(W_{T(F|\mathcal{A}^+)})$ are equivalent with $\prec_{X(\tau'_i)}=\prec_{W_{T(F|\mathcal{A}^+)}}$, by Lemma 3.3 $X(\tau'_i)$ is an execution of $\mathcal{A}^+$ and $F|\mathcal{A}^+$ is an execution of $\mathcal{A}^+$.

□

The previous lemma together with Lemmas 2.1 and 3.3 establishes that τ'_i can be transformed using Algorithm 2 into a subexecution F of the current execution E of $\mathcal{A}^+$. Consequently, testing the linearizability of $X(\tau'_i)$ suffices to verify the correctness of $\mathcal{A}^+$.

Corollary 5.3 ($X(\tau'_i)$ and the current execution)**.** *Let E be any finite execution of $\mathcal{V}^+$, and let τ'_i be the set obtained by process p_i in Line 10 in its last iteration of the while loop. There exists a finite execution F of $\mathcal{V}^+$ such that:*

$$\mathsf{LIN}(X(\tau'_i)) \iff \mathsf{LIN}(T(F|\mathcal{A}^+))$$

Proof. By Lemma 5.2, we have $\prec_{X(\tau'_i)} = \prec_{T(F|\mathcal{A}^+)}$, moreover, an execution $X(\tau'_i)$ is equivalent to the tight execution $T(F|\mathcal{A}^+)$, because for each p_i, $X(\tau'_i)|p_i = T(F|\mathcal{A}^+)|p_i$, by construction. Lemma 2.1 implies $\mathsf{LIN}(T(F|\mathcal{A}^+)) \implies \mathsf{LIN}(X(\tau'_i))$ and $\mathsf{LIN}(X(\tau'_i)) \implies \mathsf{LIN}(T(F|\mathcal{A}^+))$. □

Finally, we prove our main result, correctness of $\mathcal{V}^+$ in Theorem 5.4. For strong soundness, the proof shows that if $X(\tau'_i)$ is not linearizable, it is fine that processes report ERROR, since $X(\tau'_i)$ is an execution of $\mathcal{A}^+$. For completeness, it shows that if the current execution of $\mathcal{A}^+$ is not linearizable and, due to asynchrony, p_i locally computes an execution $X(\tau'_i)$ that is linearizable, then eventually, after a finite number of iterations of the do-while loop, p_i collects from M the non-linearizable behavior of $\mathcal{A}^+$ and hence at that moment $X(\tau'_i)$ is not linearizable.

Theorem 5.4 (Correctness of $\mathcal{V}^+$)**.** *The verifier $\mathcal{V}^+$ runtime verifies linearizability of any object O with any input algorithm $\mathcal{A}^+$. Furthermore, $\mathcal{V}^+$ satisfies the following stability property: if a process reports* ERROR *in an infinite execution E of $\mathcal{V}^+$ and $\neg\mathsf{LIN}(E'|\mathcal{A}^+)$ for some finite prefix E', then eventually every process reports* ERROR *forever.*

Proof. We argue $\mathcal{V}^+$ is a strong verifier (Definition 2.2), consider any infinite execution H of $\mathcal{V}^+$ with any arbitrary input implementation $\mathcal{A}^+$:

- Strong soundness: Consider any finite prefix E of H, such that $\mathsf{LIN}(E|\mathcal{A}^+)$.

  Consider any process p_i which obtains τ'_i in its last iteration in E, by Lemma 5.2, exists an execution F that can be obtained from E such that $\prec_{T(F|\mathcal{A}^+)}=\prec_{X(\tau'_i)}$. If $\mathsf{LIN}(T(F|\mathcal{A}^+))$ then $\mathsf{LIN}(X(\tau'_i))$, by Corollary 5.3, then no process reports ERROR and $X(\tau'_i)$ is an execution of $\mathcal{A}^+$. However, by Corollary 3.5, could happen that $\neg\mathsf{LIN}(T(E|\mathcal{A}^+))$ and if $\neg\mathsf{LIN}(T(F|\mathcal{A}^+))$, then, by Corollary 5.3 and Lemma 3.4, $\neg\mathsf{LIN}(X(\tau'_i))$ and $\neg\mathsf{LIN}(T(E|\mathcal{A}))$, then process p_i reports ERROR and $X(\tau'_i)$ is a witness of $\mathcal{A}^+$.

- Completeness: Consider any finite prefix E of H, such that $\neg\mathsf{LIN}(E|\mathcal{A}^+)$. Let $W_{T(E|\mathcal{A}^+)}$ be the set with the complete operations in the tight execution $T(E|\mathcal{A}^+)$ associated to $E|\mathcal{A}^+$.

  Consider any process p_i which obtains τ'_i in its last iteration in E, by Lemma 5.2, exists an execution F that can be obtained from E such that $\prec_{T(F|\mathcal{A}^+)}=\prec_{X(\tau'_i)}$. If $\neg\mathsf{LIN}(T(F|\mathcal{A}^+))$ then $\neg\mathsf{LIN}(X(\tau'_i))$, by Corollary 5.3, hence process p_i reports ERROR and $X(\tau'_i)$ is a witness of $\mathcal{A}^+$, by Lemma 5.2. However, if $\mathsf{LIN}(T(F|\mathcal{A}^+))$, then $\mathsf{LIN}(X(\tau'_i))$, by Corollary 5.3, hence τ'_i does not have all written operations in M, by Lemma 5.1, and process p_i does not report ERROR and $X(\tau'_i)$ is a witness of $\mathcal{A}^+$, by Lemma 5.2.

  Nevertheless, eventually some process will report ERROR. Consider any finite prefix D of H, such that E is a finite prefix of D, $\neg\mathsf{LIN}(D|\ \mathcal{A}^+)$, by the prefix-closed property of linearizability [45]. Consider any process p_i which obtains τ'_i in its last iteration in D. If all operations in $W_{T(E|\mathcal{A}^+)}$ are already in M before the p_i's collect starts, then, by Lemma 5.1, $W_{T(E|\mathcal{A}^+)} \subseteq \tau'_i$ and $\neg\mathsf{LIN}(X(\tau'_i))$, hence p_i reports ERROR and $X(\tau'_i)$ is a witness of $\mathcal{A}^+$, by Lemma 5.2.

- Stability: Consider any finite prefix E of H, such that $\neg\mathsf{LIN}(E|\mathcal{A}^+)$. Let $W_{T(E|\mathcal{A}^+)}$ be the set with the complete operations in the tight execution $T(E|\mathcal{A}^+)$ associated to $E|\mathcal{A}^+$. For every finite prefix E' of H, such that E is a finite prefix of E', $\neg\mathsf{LIN}(E'|\mathcal{A}^+)$, by the prefix-closed property of linearizability [45]. Consider any process p_i which obtains τ'_i in its last iteration in E'. If all operations in $W_{T(E|\mathcal{A}^+)}$ are already in M before the p_i's collect starts, then, by Lemma 5.1, $W_{T(E|\mathcal{A}^+)} \subseteq \tau'_i$ and $\neg\mathsf{LIN}(X(\tau'_i))$, hence p_i reports ERROR and $X(\tau'_i)$ is a witness of $\mathcal{A}^+$, by Lemma 5.2.

□

6. The Lightweight Runtime Verified $\mathcal{B}^+$

We present $\mathcal{B}^+$, a lightweight crash-tolerant runtime verified algorithm combining $\mathcal{A}^+$ and $\mathcal{V}^+$. Processes are divided into two groups: *producers*, $p_1, p_2, \ldots, p_x$, and *verifiers*, $v_1, v_2, \ldots, v_y$. Intuitively, in $\mathcal{B}^+$, producers and verifiers execute in parallel $\mathcal{A}^+$ and $\mathcal{V}^+$, with the difference that producers *do not* return views of operations. Algorithm 4 describes $\mathcal{B}^+$. Producers produce responses to operations using $\mathcal{A}$ (Lines 2 and 3), the algorithm wrapped in $\mathcal{A}^+$, and record the sketch of the current execution of $\mathcal{B}^+$ (Lines 1, 4 and 5). Verifiers are in charge of verifying, from time to time, the execution that producers have recorded so far (Lines 7 to 16).

Producers and verifiers communicate through two wait-free *decoupled* collect objects. In the spirit of known decoupled snapshot algorithms [25, 18], a decoupled collect object separates collects in two actions: obtaining a *reference* and obtaining the *actual* values. An x-dimensional decoupled collect object stores a set in each of its entries [2], and provides three operations: $Insert(i, v)$ that allows producer p_i to insert v in the i-th set, $TakeCollect()$ that allows any process to take a *reference* (an integer) of a collect of the sets, and $GetCollect(ref)$ that retrieves the actual values in the collect with reference ref. We present a simple and wait-free decoupled collect algorithm in which $Insert(i, v)$ executes two shared memory operations and $TakeCollect()$ executes a single shared memory operation.

Algorithm 4 The lightweight runtime verified linearizable algorithm $\mathcal{B}^+$.

Shared Variables:
  S, V : x-dimensional wait-free decoupled collect objects, each entry initialized to $\emptyset$

Operation Apply(op_i) **is** ▹ Code for producer p_i
1: $S.Insert(i, inv(op_i))$
2: Invoke operation Apply(op_i) of $\mathcal{A}$
3: $res(op_i) \leftarrow$ response from operation Apply(op_i) of $\mathcal{A}$
4: $ref_i \leftarrow S.TakeCollect()$ ▹ Obtains a reference to the view of op_i
5: $V.Insert(i, (p_i, inv(op_i), res(op_i), ref_i))$
6: **return** $res(op_i)$
end Apply

Operation Verify() **is** ▹ Code for verifier v_j
7: **while** *true* **do**
8: $c_j \leftarrow V.GetCollect(V.TakeCollect())$ ▹ Obtains a collect of V in two steps
9: $\tau_j \leftarrow \emptyset$
10: **for** $r_j \leftarrow 1$ up to x **do**
11: **for** each $(p_j, inv(op_i), res(op_i), ref_j) \in c_j[r_j]$ **do**
12: $\tau_j \leftarrow \tau_j \cup \{(p_i, inv(op_i), res(op_i), C.GetCollect(ref_j))\}$ ▹ Actual view of op_i
13: **end for**
14: **end for**
15: $\tau'_j \leftarrow refine(\tau_j)$
16: **if** $\neg LIN(X(\tau'_j))$ **then**
17: **report**(ERROR, $X(\tau'_j)$)
18: **end if**
19: **Sleep** for T units of time ▹ T is an application dependent constant
20: **end while**
end Verify

Algorithm 5 contains a wait-free algorithm of an x-dimensional decoupled collect object with an evergrowing set in each entry, initialized to $\emptyset$. In the implementation, only producers can invoke *Insert*, but any process can invoke *TakeCollect* and *GetCollect*.

The core idea of the algorithm is simple. A shared *Fetch&Add* variable, denoted as *CLOCK*, serves as a global clock to measure abstract time during the execution. Whenever a producer process p_i inserts a value v into its local set, it first obtains a *timestamp* for v by performing *Fetch&Add*(1) on *CLOCK* (Line 1), which atomically returns the current value and increments the clock by one. The process then stores the timestamped value in $M[i]$ (Lines 2 and 3). This mechanism allows the *TakeCollect* operation to define a collect reference simply by reading the current value of CLOCK (Line 5). The key observation is that all *Insert* operations that precede a *TakeCollect* operation, say op, which reads value t from *CLOCK*, must have inserted their elements with timestamps strictly less than t. In contrast, *Insert* operations that are concurrent to op may receive timestamps greater than or equal to t, the latter occurs if op reads *CLOCK* before a concurrent

[2] We focus on the case that each entry stores an evergrowing set, but the definition can be generalized.

Algorithm 5 A wait-free x-dimensional decoupled collect algorithm.

Shared Variables:
M : x-dimensional wait-free collect object, with each entry initialized to $\emptyset$
$CLOCK$: $Fecth\&Add$ atomic object, initialized to 0.

Local Persistent Variable:
$uset_i$: unbounded set, initialized to $\emptyset$. ▹ Local variables of producer p_i

Operation Insert(i, v) **is** ▹ Code for producer p_i
1: $time_i \leftarrow CLOCK.Fetch\&Add(1)$
2: $uset_i \leftarrow uset_i \cup \{(time_i, v)\}$
3: $M.Write(i, uset_i)$
4: **return** OK
end Insert

Operation TakeCollect() **is** ▹ Code for any process
5: **return** $CLOCK.Read()$
end TakeCollect

Operation GetCollect(ref) **is** ▹ Code for any process
6: $c_i[1, \ldots, x] \leftarrow [\emptyset, \ldots, \emptyset]$
7: $m_i \leftarrow M.Collect()$
8: **for** $r_i \leftarrow 1$ up to x **do**
9: **for** each $(t_i, v_i) \in m_i[r_i]$ with $t_i \leq ref$ **do**
10: $c_i[r_i] \leftarrow c_i[r_i] \cup \{v_i\}$
11: **return** c_i
end GetCollect

Insert accesses $CLOCK$. Consequently, op is guaranteed to read all prior *Insert* operations and possibly some concurrent ones, which is precisely the definition of the collect operation. To retrieve the actual values associated with a collect reference ref, Lines 6 to 11 perform a two step procedure: the contents of all sets in M are collected, and then any values with timestamps greater to ref are discarded.

The following theorem assumes that there is at least one correct producer and at least one correct verifier in any execution of $\mathcal{B}^+$.

Theorem 6.1 (Correctness of $\mathcal{B}^+$)**.** *Let $\mathcal{A}$ be any algorithm that is presumably linearizable, and consider its related runtime verified algorithm $\mathcal{B}^+$ as defined above. Then,*
1) $\mathcal{B}^+$ is linearizable if and only if $\mathcal{A}$ is linearizable;
2) $\mathcal{B}^+$ is wait-free (resp. lock-free) if and only if $\mathcal{A}$ is wait-free (resp. lock-free);
3) $\mathcal{B}^+$'s operation step complexity is $\mathcal{A}$'s step complexity plus five;
4) in $\mathcal{B}^+$'s executions, if a verifier reports ERROR*, it provides a non-linearizable execution of $\mathcal{B}^+$, and if the current execution is not linearizable, eventually all correct verifiers report* ERROR *forever.*

Proof. Items 1) and 2) follow directly from Lemma 3.2. Item 3) follows from the fact that in the decoupled collect algorithm above, *Insert* performs two shared memory operations, and $TakeCollect$ performs one. Regarding item 4), the key observation is that producers are basically executing $\mathcal{A}^+$, with the difference that they do not return their views, they record them in the decoupled collect object S. As a result, S stores an sketch of the current execution of $\mathcal{B}^+$. Thus, item 4) follows from Theorem 5.4, and from the assumption that there are at least one correct producer and one correct verifier in every execution.

To avoid redundancy with earlier arguments, we briefly outline the essential steps required to establish the implication from Theorem 5.4:

1) Constructing a Tight Execution: Consider any execution E in which n producers invoke $apply()$ operations and m verifiers invoke $verify()$ operations. The steps in Algorithm 5 (Line 3) are analogous to those in Algorithm 1 (Line 2). Similarly, the collect operation in Algorithm 1 (Line 5) is now represented by Algorithm 4 (Line 5). Lemma 3.4 holds because the shared operations in Algorithm 4 strengthen the interval of the $apply()$ operation.

2) Well-Formed Execution: From the views collected by the verifiers, we can construct a well-formed execution that satisfies the properties of Theorem 5.4. This is primarily because the properties in Lemma 4.1 hold. However, we must now account for the $CLOCK$ mechanism.

3) Properties of Collected Views: Consider any execution E where n producers invoke $apply()$ operations and m verifiers periodically invoke $verify()$ operations. Let c_i and c_j be the views obtained by verifiers v_i and v_j (possibly with $v_i = v_j$) with inputs ref_i and ref_j in Line 11 of Algorithm 5. Let ref_a and ref_b be the references shared by producers p_a and p_b in Line 5 of Algorithm 4, corresponding to operations op_a and op_b.

⋄ Self-Inclusion Property: If $(a, \text{inv}(op_a))$ is inserted in Line 1 of Algorithm 4 and $ref_a < ref_i$, then $ref_a \geq a$ and $op_a \in c_i$. This ensures that the self-inclusion property holds.

⋄ Containment in Precedence Property: If $op_a \prec_E op_b$, and if $op_b \in c_i$, then $op_a \in c_i$ or if $op_b \in c_j$, then $op_a \in c_j$. This ensures that the containment in precedence property holds.

⋄ Non-Empty Intersection Property: If $op_a \parallel_E op_b$ and $ref_a, ref_b \geq ref_i, ref_j$, then $m_i \cap m_j \neq \emptyset$. This ensures that the non-empty intersection property holds.

These three properties guarantee that the collected views form a sketch of the current execution satisfying Theorem 4.9, and thereby fulfilling the structural requirements of Theorem 5.4.

□

Corollary 6.2 ($\mathcal{B}^+$ with only read/write operations)**.** *The algorithm $\mathcal{B}^+$ preserves all properties stated in Theorem 6.1 even when using the relaxed $Fetch\&add$ operation from [24], which:*

- *Requires only read/write operations.*
- *Allows distinct operations to obtain the same value, but only if they are concurrent.*
- *Can safely replace the standard $Fetch\&add$ in the $CLOCK$ implementation.*

Proof. Items 1), 2), and 3) hold directly from Theorem 6.1. The relaxed $fetch\&add$ operation only introduces additional read/write operations, which do not affect the linearizability, the wait-free property, or step complexity of $\mathcal{B}^+$. The key observation for item 4) is that the relaxed $fetch\&add$ operation allows concurrent operations, Lines 1 and 5 of Algorithm 4, to obtain the same value, but this does not affect the construction of a tight execution or the properties of the collected views. Specifically: The construction of the tight execution remains the same as in Theorem 6.1. The relaxed $fetch\&add$ operation only affects the values assigned to concurrent operations,

but it does not alter the order or structure of the execution. The properties of the collected views (self-inclusion, containment in precedence, and non-empty intersection) remain unchanged. This is because the relaxed $fetch\&add$ operation does not interfere with the logical relationships between operations or the views collected by verifiers.

□

7. Discussion

7.1. Contribution

We enhance the theoretical Verifier $\mathcal{V}^*$ and the Wrapper $\mathcal{A}^*$ from [12] by reading all array entries non-atomically. This modification simplifies the communication interface, shifting the computational complexity to the verification interface.

As a result, our improvements enable lightweight and practical runtime verification implementations, moving beyond the snapshot-based approach, which remains largely theoretical. A concrete instance of such a lightweight verifier is $\mathcal{B}^+$, which relies solely on the atomic operation `Fetch&Add`. Moreover, this operation can be replaced by a relaxed, set-linearizable variant that uses only read/write operations, as shown in [24].

Building upon the model from [12], our work introduces a refined definition that focuses on issuing a verdict based on the detected execution. In addition, we provide an explicit analysis of the discrepancies between the actual execution and the detected one, an aspect that previous works [46, 47, 41] have not addressed.

Another central contribution of our approach is the use of *interval detection* to characterize execution behavior. This technique is generally overlooked in the context of distributed asynchronous systems. Moreover, while prior works that verify linearizability for a single execution rely on the notion of intervals, these are typically implicit and derived solely from invocation/response pairs, without the explicit detection mechanisms developed in our work [9, 10, 5, 41].

7.2. Indirect Verification and Weaker Memory Models

Despite these improvements, our approach still relies on atomicity for writes and individual reads. This raises the following question: Can we employ a weaker memory model, such as sequential consistency, to achieve distributed runtime verification of linearizability more efficiently? The answer is *No*, because we require stronger guarantees regarding the time at which some writes take effect, for example, the write in Line 2 in Algorithm 1. In particular, under a sequentially consistent memory model, writes from different processes can be observed in different orders by different readers, meaning that the execution reconstructed by the Verifier $\mathcal{V}^+$ might not correspond to a single global order, violating the constraints imposed by Definition 2.2.

Furthermore, due to the non-composability of sequential consistency, even if the underlying algorithm $\mathcal{A}$ is linearizable, the combined system, including both $\mathcal{A}^+$ and $\mathcal{V}^+$, might fail to be sequentially consistent.

To illustrate this issue, consider a two-process execution E of $\mathcal{V}^+$ in Algorithm 3, where $\mathcal{A}^+$ is executed over a sequentially consistent memory model. Let op_i and op_j be two complete operations in $E|\mathcal{A}^+$, invoked by processes p_i and p_j, respectively.

If op_i and op_j are concurrent in $E|\mathcal{A}^+$, i.e., $op_i \parallel_{E|\mathcal{A}^+} op_j$, the tight execution equivalent obtained by the verifier may yield $op_j \prec_{T(E)} op_i$, even if $op_i \prec_{E|\mathcal{A}^+} op_j$. This discrepancy violates the second item of Lemma 3.4. Such a scenario arises if the write line by process p_i takes effect after process p_j has already started its collect operation in Line 5 of Algorithm 1, as illustrated in Figure 7.

A tight execution represents the execution as perceived by processes through write and collect operations on the shared array. However, in this case, the resulting tight execution does not correspond to an execution of $\mathcal{A}^+$, because we can no longer construct an execution $T(E)$ from E by simply removing local steps of the processes, as required by Lemma 3.3.

Additionally, since any process can obtain a sketch of the current execution (e.g., $op_j \prec_{X(\tau')} op_i$) and check its linearizability, this discrepancy can lead to incorrect verification results. Specifically, if we assume that $\mathsf{LIN}(\mathcal{A})$ holds, meaning that an execution of $\mathcal{A}$ satisfies $op_j \prec op_i$, but the reconstructed execution $op_i \prec op_j$ is not linearizable, then we obtain that $\mathsf{LIN}(\mathcal{A})$ holds, but $\neg\mathsf{LIN}(\mathcal{A}^+)$ is detected, as result contradicting Strong Soundness (Definition 2.2).

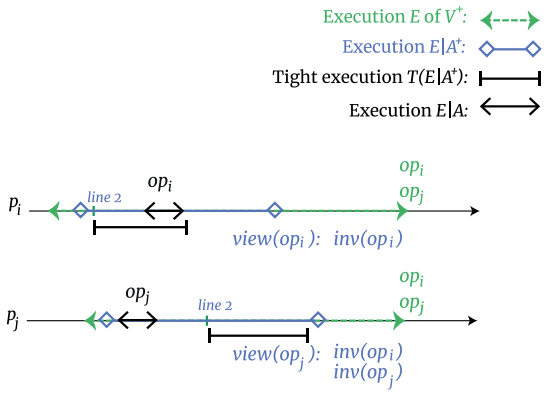


Figure 7: Example of an execution of $\mathcal{V}^+$ and $\mathcal{A}^+$ over a sequentially consistent model

We choose a sequential consistency model, as it is considered a strong correctness condition and a safety property, similar to linearizability, yet weaker than it [3]. Moreover, the verification model and algorithms presented here are sufficient to detect the current execution and verify sequential consistency. However, just as with linearizability, sequential consistency is also impossible to verify asynchronously at runtime [12].

The relationship between sequential consistency and linearizability has been extensively studied [48, 49]. The idea of using a linearizable algorithm over a sequentially consistent memory model is not new. In [49], the authors provide an insightful perspective, demonstrating that in fully asynchronous systems, linearizable and sequentially consistent implementations are indistinguishable. Specifically, they prove that executing a linearizable implementation in a sequential consistency

model yields a sequentially consistent and composable implementation.

7.3. Further Work

We advocate for the use of simple and efficient communication interfaces, as demonstrated by collects, as a guiding principle for designing streamlined interaction mechanisms. This approach is exemplified in the $\mathcal{B}^+$ algorithm, where communication relies solely on primitives with a consensus number of at most 2, or even on read/write operations alone.

A promising direction for future research involves exploring the limitations of interval detection under weaker memory models. We conjecture that, in models such as sequential consistency, the interval detection technique introduced in [12] becomes ineffective. Our intuition is that approximate real-time order detection critically depends on atomicity: without it, vital ordering information is lost, and no guarantees exist about when operations actually take effect. As a result, notions such as stretched intervals and the tight execution abstraction from Section 3 are no longer valid.

Although the interval-based verification model extends beyond linearizability, supporting sequential and eventual consistency as discussed in [36], we emphasize the need for a software-level analysis of the cost-benefit tradeoffs involved in applying interval detection within atomic memory models. Such an analysis would help assess the practicality and scalability of these techniques in real-world systems.

References

[1] M. Leucker, C. Schallhart, A brief account of runtime verification, J. Log. Algebraic Methods Program. 78 (2009) 293–303. doi:10.1016/j.jlap.2008.08.004.
URL https://doi.org/10.1016/j.jlap.2008.08.004

[2] B. Bonakdarpour, P. Fraigniaud, S. Rajsbaum, C. Travers, Challenges in fault-tolerant distributed runtime verification, in: T. Margaria, B. Steffen (Eds.), Leveraging Applications of Formal Methods, Verification and Validation: Discussion, Dissemination, Applications - 7th International Symposium, ISoLA 2016, Imperial, Corfu, Greece, October 10-14, 2016, Proceedings, Part II, Vol. 9953 of Lecture Notes in Computer Science, 2016, pp. 363–370. doi:10.1007/978-3-319-47169-3_27.
URL https://doi.org/10.1007/978-3-319-47169-3_27

[3] M. Herlihy, J. M. Wing, Linearizability: A correctness condition for concurrent objects, ACM Trans. Program. Lang. Syst. 12 (3) (1990) 463–492. doi:10.1145/78969.78972.
URL https://doi.org/10.1145/78969.78972

[4] P. B. Gibbons, E. Korach, Testing shared memories, SIAM J. Comput. 26 (4) (1997) 1208–1244. doi:10.1137/S0097539794279614.
URL https://doi.org/10.1137/S0097539794279614

[5] A. Bouajjani, M. Emmi, C. Enea, J. Hamza, Tractable refinement checking for concurrent objects, in: S. K. Rajamani, D. Walker (Eds.), Proceedings of the 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2015, Mumbai, India, January 15-17, 2015, ACM, 2015, pp. 651–662. doi:10.1145/2676726.2677002.
URL https://doi.org/10.1145/2676726.2677002

[6] A. Bouajjani, M. Emmi, C. Enea, J. Hamza, On reducing linearizability to state reachability, Information and Computation 261 (2018) 383–400, iCALP 2015. doi:https://doi.org/10.1016/j.ic.2018.02.014.
URL https://www.sciencedirect.com/science/article/pii/S0890540118300178

[7] M. Emmi, C. Enea, Symbolic abstract data type inference, SIGPLAN Not. 51 (1) (2016) 513–525. doi:10.1145/2914770.2837645.
URL https://doi.org/10.1145/2914770.2837645

[8] M. Emmi, C. Enea, Sound, complete, and tractable linearizability monitoring for concurrent collections, Proc. ACM Program. Lang. 2 (POPL) (2018) 25:1–25:27. doi:10.1145/3158113.
URL https://doi.org/10.1145/3158113

[9] T. Elmas, S. Tasiran, S. Qadeer, VYRD: verifying concurrent programs by runtime refinement-violation detection, in: V. Sarkar, M. W. Hall (Eds.), Proceedings of the ACM SIGPLAN 2005 Conference on Programming Language Design and Implementation, Chicago, IL, USA, June 12-15, 2005, ACM, 2005, pp. 27–37. doi:10.1145/1065010.1065015.
URL https://doi.org/10.1145/1065010.1065015

[10] T. Elmas, S. Tasiran, Vyrdmc: Driving runtime refinement checking with model checkers, in: H. Barringer, B. Finkbeiner, Y. Gurevich, H. Sipma (Eds.), Proceedings of the Fifth Workshop on Runtime Verification, RV@CAV 2005, Edinburgh, UK, July 12, 2005, Vol. 144 of Electronic Notes in Theoretical Computer Science, Elsevier, 2005, pp. 41–56. doi:10.1016/j.entcs.2006.02.003.
URL https://doi.org/10.1016/j.entcs.2006.02.003

[11] B. Bonakdarpour, P. Fraigniaud, S. Rajsbaum, D. A. Rosenblueth, C. Travers, Decentralized asynchronous crash-resilient runtime verification, J. ACM 69 (5) (2022) 34:1–34:31. doi:10.1145/3550483.
URL https://doi.org/10.1145/3550483

[12] A. Castañeda, G. V. Rodríguez, Asynchronous wait-free runtime verification and enforcement of linearizability, in: R. Oshman, A. Nolin, M. M. Halldórsson, A. Balliu (Eds.), Proceedings of the 2023 ACM Symposium on Principles of Distributed Computing, PODC 2023, Orlando, FL, USA, June 19-23, 2023, ACM, 2023, pp. 90–101. doi:10.1145/3583668.3594563.
URL https://doi.org/10.1145/3583668.3594563

[13] L. Lamport, Time, clocks, and the ordering of events in a distributed system, Commun. ACM 21 (7) (1978) 558–565. doi:10.1145/359545.359563.
URL https://doi.org/10.1145/359545.359563

[14] P. Kuznetsov, R. Guerraoui, Algorithms for Concurrent Systems, EPFL Press, 2018.

[15] H. Attiya, A. Bar-Noy, D. Dolev, Sharing memory robustly in message-passing systems, J. ACM 42 (1) (1995) 124–142. doi:10.1145/200836.200869.
URL https://doi.org/10.1145/200836.200869

[16] H. Attiya, F. Ellen, P. Fatourou, The complexity of updating snapshot objects, Journal of Parallel and Distributed Computing 71 (12) (2011) 1570–1577. doi:https://doi.org/10.1016/j.jpdc.2011.08.002.
URL https://www.sciencedirect.com/science/article/pii/S0743731511001596

[17] M. Inoue, W. Chen, Linear-time snapshot using multi-writer multi-reader registers, in: G. Tel, P. M. B. Vitányi (Eds.), Distributed Algorithms, 8th International Workshop, WDAG '94, Terschelling, The Netherlands, September 29 - October 1, 1994, Proceedings, Vol. 857 of Lecture Notes in Computer Science, Springer, 1994, pp. 130–140. doi:10.1007/BFb0020429.
URL https://doi.org/10.1007/BFb0020429

[18] Y. Wei, N. Ben-David, G. E. Blelloch, P. Fatourou, E. Ruppert, Y. Sun, Constant-time snapshots with applications to concurrent data structures, in: J. Lee, E. Petrank (Eds.), PPoPP '21: 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, Virtual Event, Republic of Korea, February 27- March 3, 2021, ACM, 2021, pp. 31–46. doi:10.1145/3437801.3441602.
URL https://doi.org/10.1145/3437801.3441602

[19] Y. Afek, H. Attiya, D. Dolev, E. Gafni, M. Merritt, N. Shavit, Atomic snapshots of shared memory, J. ACM 40 (4) (1993) 873–890. doi:10.1145/153724.153741.
URL https://doi.org/10.1145/153724.153741

[20] E. Bartocci, Y. Falcone (Eds.), Lectures on Runtime Verification - Introductory and Advanced Topics, Vol. 10457 of Lecture Notes in Computer Science, Springer, 2018. doi:10.1007/978-3-319-75632-5.
URL https://doi.org/10.1007/978-3-319-75632-5

[21] S. Berkovich, B. Bonakdarpour, S. Fischmeister, Runtime verification with minimal intrusion through parallelism, Formal Methods Syst. Des. 46 (3) (2015) 317–348. doi:10.1007/s10703-015-0226-3.
URL https://doi.org/10.1007/s10703-015-0226-3

[22] N. Decker, P. Gottschling, C. Hochberger, M. Leucker, T. Scheffel, M. Schmitz, A. Weiss, Rapidly adjustable non-intrusive online monitor-

ing for multi-core systems, in: S. A. da Costa Cavalheiro, J. L. Fiadeiro (Eds.), Formal Methods: Foundations and Applications - 20th Brazilian Symposium, SBMF 2017, Recife, Brazil, November 29 - December 1, 2017, Proceedings, Vol. 10623 of Lecture Notes in Computer Science, Springer, 2017, pp. 179–196. doi:10.1007/978-3-319-70848-5_12.
URL https://doi.org/10.1007/978-3-319-70848-5_12

[23] C. Soueidi, A. El-Hokayem, Y. Falcone, Opportunistic monitoring of multithreaded programs, in: L. Lambers, S. Uchitel (Eds.), Fundamental Approaches to Software Engineering - 26th International Conference, FASE 2023, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2023, Paris, France, April 22-27, 2023, Proceedings, Vol. 13991 of Lecture Notes in Computer Science, Springer, 2023, pp. 173–194. doi:10.1007/978-3-031-30826-0_10.
URL https://doi.org/10.1007/978-3-031-30826-0_10

[24] A. Castañeda, S. Rajsbaum, M. Raynal, Set-linearizable implementations from read/write operations: Sets, fetch &increment, stacks and queues with multiplicity, Distributed Computing 36 (2) (2023) 89–106. doi:10.1007/s00446-022-00440-y.
URL https://doi.org/10.1007/s00446-022-00440-y

[25] B. Bashari, P. Woelfel, An efficient adaptive partial snapshot implementation, in: A. Miller, K. Censor-Hillel, J. H. Korhonen (Eds.), PODC '21: ACM Symposium on Principles of Distributed Computing, Virtual Event, Italy, July 26-30, 2021, ACM, 2021, pp. 545–555. doi:10.1145/3465084.3467939.
URL https://doi.org/10.1145/3465084.3467939

[26] J. M. Lourenço, J. Fiedor, B. Krena, T. Vojnar, Discovering concurrency errors, in: E. Bartocci, Y. Falcone (Eds.), Lectures on Runtime Verification - Introductory and Advanced Topics, Vol. 10457 of Lecture Notes in Computer Science, Springer, 2018, pp. 34–60. doi:10.1007/978-3-319-75632-5_2.
URL https://doi.org/10.1007/978-3-319-75632-5_2

[27] M. Herlihy, N. Shavit, The art of multiprocessor programming, Morgan Kaufmann, 2008.

[28] M. Moir, N. Shavit, Concurrent data structures, in: D. P. Mehta, S. Sahni (Eds.), Handbook of Data Structures and Applications, Chapman and Hall/CRC, 2004. doi:10.1201/9781420035179.ch47.
URL https://doi.org/10.1201/9781420035179.ch47

[29] M. Raynal, Concurrent Programming - Algorithms, Principles, and Foundations, Springer, 2013. doi:10.1007/978-3-642-32027-9.
URL https://doi.org/10.1007/978-3-642-32027-9

[30] L. Lamport, How to make a multiprocessor computer that correctly executes multiprocess programs, IEEE Trans. Computers 28 (9) (1979) 690–691. doi:10.1109/TC.1979.1675439.
URL https://doi.org/10.1109/TC.1979.1675439

[31] P. Fraigniaud, S. Rajsbaum, C. Travers, A lower bound on the number of opinions needed for fault-tolerant decentralized run-time monitoring, J. Appl. Comput. Topol. 4 (1) (2020) 141–179. doi:10.1007/s41468-019-00047-6.
URL https://doi.org/10.1007/s41468-019-00047-6

[32] Y. Falcone, On decentralized monitoring, in: A. Nouri, W. Wu, K. Barkaoui, Z. Li (Eds.), Verification and Evaluation of Computer and Communication Systems - 15th International Conference, VECoS 2021, Virtual Event, November 22-23, 2021, Revised Selected Papers, Vol. 13187 of Lecture Notes in Computer Science, Springer, 2021, pp. 1–16. doi:10.1007/978-3-030-98850-0_1.
URL https://doi.org/10.1007/978-3-030-98850-0_1

[33] A. Francalanza, J. A. Pérez, C. Sánchez, Runtime verification for decentralised and distributed systems, in: E. Bartocci, Y. Falcone (Eds.), Lectures on Runtime Verification - Introductory and Advanced Topics, Vol. 10457 of Lecture Notes in Computer Science, Springer, 2018, pp. 176–210. doi:10.1007/978-3-319-75632-5_6.
URL https://doi.org/10.1007/978-3-319-75632-5_6

[34] C. Sánchez, G. Schneider, W. Ahrendt, E. Bartocci, D. Bianculli, C. Colombo, Y. Falcone, A. Francalanza, S. Krstic, J. M. Lourenço, D. Nickovic, G. J. Pace, J. Rufino, J. Signoles, D. Traytel, A. Weiss, A survey of challenges for runtime verification from advanced application domains (beyond software), Formal Methods Syst. Des. 54 (3) (2019) 279–335. doi:10.1007/s10703-019-00337-w.
URL https://doi.org/10.1007/s10703-019-00337-w

[35] G. V. Rodríguez, A. Castañeda, Towards efficient runtime verified linearizable algorithms, in: Runtime Verification: 24th International Conference, RV 2024, Istanbul, Turkey, October 15–17, 2024, Proceedings, Springer-Verlag, Berlin, Heidelberg, 2024, p. 262–281. doi:10.1007/978-3-031-74234-7_17.
URL https://doi.org/10.1007/978-3-031-74234-7_17

[36] A. Castañeda, G. V. Rodríguez, Asynchronous fault-tolerant language decidability for runtime verification of distributed systems (2025). arXiv:2502.00191.
URL https://arxiv.org/abs/2502.00191

[37] A. Pnueli, A. Zaks, Psl model checking and run-time verification via testers, in: Proceedings of the 14th International Conference on Formal Methods, FM'06, Springer-Verlag, Berlin, Heidelberg, 2006, p. 573–586. doi:10.1007/11813040_38.
URL https://doi.org/10.1007/11813040_38

[38] R. Taleb, S. Hallé, R. Khoury, Uncertainty in runtime verification: A survey, Comput. Sci. Rev. 50 (2023) 100594. doi:10.1016/J.COSREV.2023.100594.
URL https://doi.org/10.1016/j.cosrev.2023.100594

[39] M. Herlihy, D. N. Kozlov, S. Rajsbaum, Distributed Computing Through Combinatorial Topology, Morgan Kaufmann, 2013.
URL https://store.elsevier.com/product.jsp?isbn=9780124045781

[40] P. W. O'Hearn, N. Rinetzky, M. T. Vechev, E. Yahav, G. Yorsh, Verifying linearizability with hindsight, in: Proceedings of the 29th ACM SIGACT-SIGOPS Symposium on Principles of Distributed Computing, PODC '10, Association for Computing Machinery, New York, NY, USA, 2010, p. 85–94. doi:10.1145/1835698.1835722.
URL https://doi.org/10.1145/1835698.1835722

[41] M. Emmi, C. Enea, Violat: Generating tests of observational refinement for concurrent objects, in: I. Dillig, S. Tasiran (Eds.), Computer Aided Verification, Springer International Publishing, Cham, 2019, pp. 534–546.

[42] J. Deshmukh, G. Ramalingam, V.-P. Ranganath, K. Vaswani, Logical concurrency control from sequential proofs, in: A. D. Gordon (Ed.), Programming Languages and Systems, Springer Berlin Heidelberg, Berlin, Heidelberg, 2010, pp. 226–245.

[43] N. A. Lynch, F. W. Vaandrager, Forward and backward simulations: I. untimed systems, Inf. Comput. 121 (2) (1995) 214–233. doi:10.1006/inco.1995.1134.
URL https://doi.org/10.1006/inco.1995.1134

[44] M. Herlihy, Wait-free synchronization, ACM Trans. Program. Lang. Syst. 13 (1) (1991) 124–149. doi:10.1145/114005.102808.
URL https://doi.org/10.1145/114005.102808

[45] R. Guerraoui, E. Ruppert, Linearizability is not always a safety property, in: G. Noubir, M. Raynal (Eds.), Networked Systems - Second International Conference, NETYS 2014, Marrakech, Morocco, May 15-17, 2014. Revised Selected Papers, Vol. 8593 of Lecture Notes in Computer Science, Springer, 2014, pp. 57–69. doi:10.1007/978-3-319-09581-3_5.
URL https://doi.org/10.1007/978-3-319-09581-3_5

[46] A. Bauer, Y. Falcone, Decentralised LTL monitoring, Formal Methods Syst. Des. 48 (1-2) (2016) 46–93. doi:10.1007/s10703-016-0253-8.
URL https://doi.org/10.1007/s10703-016-0253-8

[47] A. El-Hokayem, Y. Falcone, Can we monitor all multithreaded programs?, in: C. Colombo, M. Leucker (Eds.), Runtime Verification - 18th International Conference, RV 2018, Limassol, Cyprus, November 10-13, 2018, Proceedings, Vol. 11237 of Lecture Notes in Computer Science, Springer, 2018, pp. 64–89. doi:10.1007/978-3-030-03769-7_6.
URL https://doi.org/10.1007/978-3-030-03769-7_6

[48] H. Attiya, J. L. Welch, Sequential consistency versus linearizability, ACM Trans. Comput. Syst. 12 (2) (1994) 91–122. doi:10.1145/176575.176576.
URL https://doi.org/10.1145/176575.176576

[49] M. Perrin, M. Petrolia, A. Mostéfaoui, C. Jard, On composition and implementation of sequential consistency, in: C. Gavoille, D. Ilcinkas (Eds.), Distributed Computing, Springer Berlin Heidelberg, Berlin, Heidelberg, 2016, pp. 284–297.